# Pattern Emergence in Drone Swarms Using Graph-Based AI Models


### Supervisor :

Dr. Khadli Belkacem

### Authors:

Amir Benslaimi
Akram Rahali
Riham Messaoudi
Aimen Abdesselam
Bouaouadja Mohammed


## 1. Introduction

Unmanned Aerial Vehicle (UAV) swarms represent a significant advancement in autonomous robotics, offering scalable and distributed solutions for complex missions such as search and rescue, structural inspection, and dynamic coverage. However, orchestrating dozens of fast-moving, physically interacting drones in cluttered environments exposes the limits of traditional, centralized architectures, which suffer from severe computational bottlenecks, communication latencies, and single points of failure. To achieve true autonomy, next-generation swarms must rely on decentralized, localized intelligence where individual agents make real-time decisions based solely on onboard sensing and peer-to-peer communication. This work presents a comprehensive, closed-loop machine learning framework designed to solve these interconnected challenges. The architecture spans a highly parallelized, physics-accurate dataset generator built in PyFlyt to produce expert flight trajectories at scale; a Graph Neural Network (GNN) policy that leverages localized dynamic attention for setpoint prediction; an implicit residual correction mechanism to smooth local multi-agent congestion and obstacle interactions; and a decentralized formation-assignment protocol that eliminates the need for a global coordinator. By fusing geometric deep learning with distributed optimization, this pipeline enables multi-drone swarms to safely coordinate, adapt their topology, and execute complex maneuvers autonomously under realistic physical constraints.

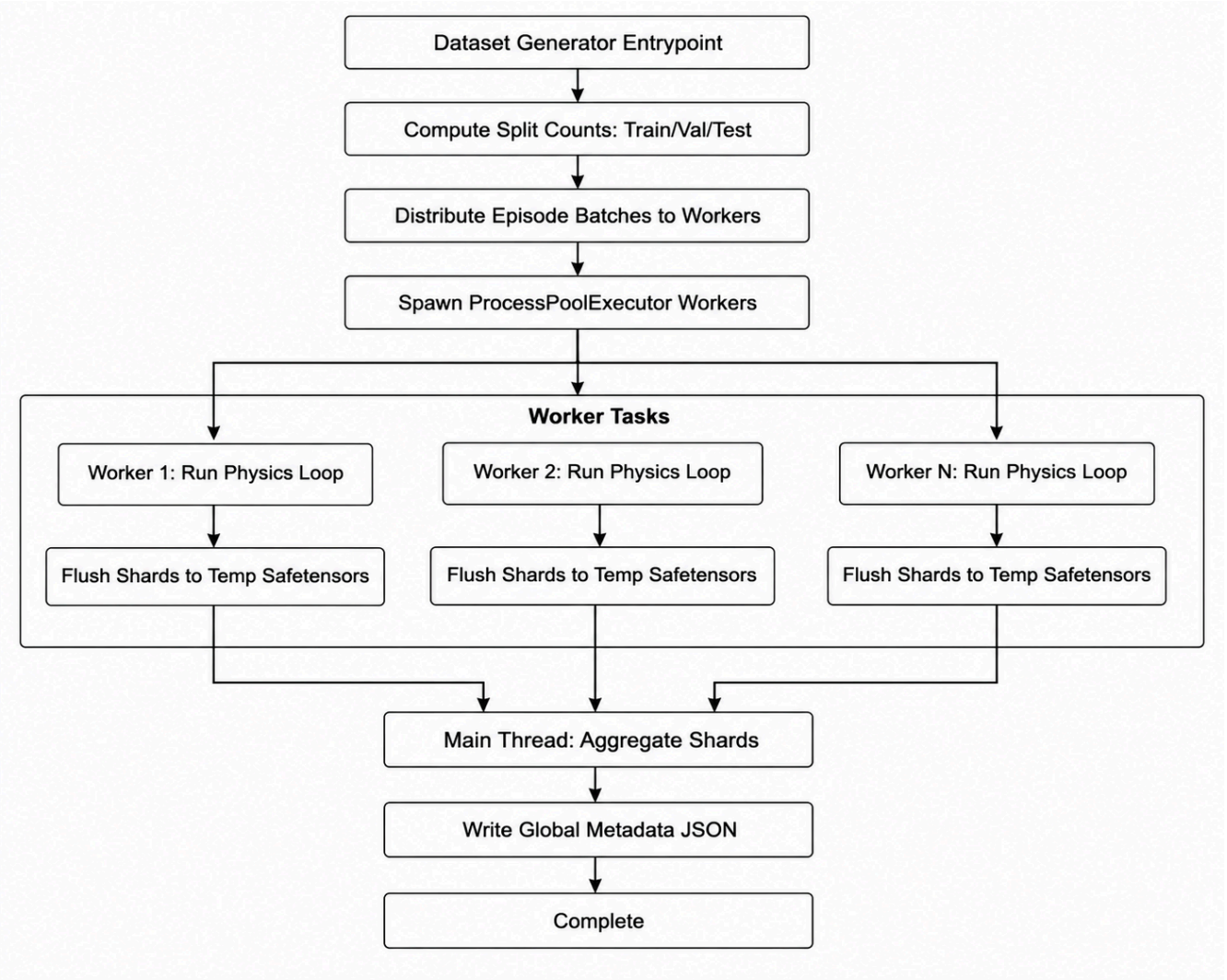
---

## 2. Dataset Collection

### 2.1 System Architecture

The dataset generator is architected for high-throughput, memory-safe parallel execution. Due to the Python Global Interpreter Lock (GIL), physics simulations cannot be effectively multi-threaded.

Therefore, the system relies on a **ProcessPoolExecutor** multiprocessing strategy.




Spawning Python worker processes introduces static overhead (importing heavy packages like `torch` and PyBullet, establishing IPC channels). For small datasets, this overhead dominates. However, for large-scale generations (100+ episodes), the physics computation time far outweighs the spawning cost, allowing the architecture to achieve near-linear multi-core scaling.

*Orchestrating concurrent simulation chunks (`dataset_generator/parallel.py`):*

```
print(f"Launching {len(worker_tasks)} worker tasks across {num_workers} parallel
processes...")
with ProcessPoolExecutor(max_workers=num_workers) as executor:
    futures = {executor.submit(simulate_worker_chunk, task): task for task in
worker_tasks}
    # ... progress tracking and Safetensor aggregation ...
```

## 2.2 Physics Simulation

The core physics loop is executed by the `run_physics_episode` function in `simulation.py`. This function initializes the PyFlyt `Aviary` and steps through time, generating expert trajectories.

### Environment Parameters and Disturbances

To ensure the IL agent learns robust policies, domain randomization is applied by injecting environmental disturbances such as physical wind and spherical obstacles. Sensor noise is also added to mimic real-world hardware (e.g., IMU drift, GPS inaccuracies).

*Domain randomization via Gaussian perturbation ( `dataset_generator/features.py` ):*

```
def maybe_add_sensor_noise(global_pos, global_euler, local_lin_vel, local_ang_vel,
noisy_sensors, noise_variance):
    if not noisy_sensors: return global_pos, global_euler, local_lin_vel,
local_ang_vel
    global_pos = global_pos + np.random.normal(0, noise_variance, size=3)
    global_euler = global_euler + np.random.normal(0, noise_variance, size=3)
    local_lin_vel = local_lin_vel + np.random.normal(0, noise_variance, size=3)
    local_ang_vel = local_ang_vel + np.random.normal(0, noise_variance, size=3)
    return global_pos, global_euler, local_lin_vel, local_ang_vel
```

## Physics vs. Control Frequency

Quadcopters are governed by highly non-linear, high-speed aerodynamic rotor forces. Simulating these ODEs requires small time steps. However, flight control units (FCUs) receive high-level waypoint commands at much lower frequencies. The simulation therefore decouples the two rates.

*Decoupling control and physics integrations ( `dataset_generator/environment.py` ):*

```
drone_options = dict()
drone_options["control_hz"] = 60  # Match physical FCU target command latencies
# Step physics engine at 240 Hz to maintain rigid-body integration stability
env = Aviary(start_pos=start_pos, start_orn=start_orn, drone_type="quadx",
             render=graphical, physics_hz=240, drone_options=drone_options)
```

## 3D Artificial Potential Field (APF)

To generate expert ground truth for trajectory navigation, safe collision-free intermediate setpoints are computed dynamically using a 3D Artificial Potential Field. The virtual force $\mathbf{F}_{total}$ is the sum of attractive target forces and repulsive obstacle forces, dynamically bounded to prevent physical impossibilities.

*APF velocity bounds ( `dataset_generator/environment.py` ):*

```
# 1. Calculate attractive vectors with horizontal/vertical specific gains
attractive = final_target_positions - current_positions
attractive[:, :2] *= attractive_gain
attractive[:, 2] *= vertical_gain

# ... [Calculate Repulsive Forces based on radial distance inside boundaries] ...

# 2. Velocity Bounding: Ensure delta movement doesn't exceed maximum kinematic
capabilities
total_delta = np.copy(attractive)
total_delta[:, :3] += repulsive_xyz
norm = np.linalg.norm(total_delta, axis=1, keepdims=True)
scale = np.minimum(1.0, max_step_size / (norm + 1e-6))
bounded_delta = total_delta * scale

# 3. Yield the safe intermediate step
intermediate_positions = current_positions + bounded_delta
```

## 2.3 Node and Edge Features

If absolute $x, y, z$ global coordinates were used, the GNN would memorize specific map locations and fail to generalize — a phenomenon known as **State Aliasing**. This is eliminated by decoupling global states into body-centric local representations and providing LiDAR-style spatial footprints.

*Decoupling representations to enforce translation/rotation invariance (`dataset_generator/features.py`):*

```
# Extract purely local kinematics and sensory inputs
gnn_input_state = np.concatenate([local_lin_vel, local_ang_vel, obs_features])

# Convert global absolute error into the drone's local coordinate system using its
rotation matrix
global_pos_error = target_global_pos - global_pos
local_pos_error = rot_matrix.T @ global_pos_error
yaw_error = target_global_yaw - global_euler[2]

# The neural network predicts this local correction vector
y_label = np.concatenate([local_pos_error, [yaw_error]])
```

## 2.4 Tapered Sampling and Convergence Stopping

- **Convergence Stopping**: If all drones remain within a `conv_threshold` (e.g., 0.2 m) of their target for more than 50 steps, the episode terminates early to avoid flooding the dataset with static data.
- **Tapered Sampling**: Dynamic stride sampling adjusts the density of recorded steps depending on the learning objective:
  - For **better convergence policies**, sampling density is increased at the beginning of episodes to capture high-dynamic transients when drones surge toward target formations.
  - For **better steady-state stability**, sampling is increased toward the end of episodes to enrich the dataset with fine-grained micro-adjustments.

*Evaluating sampling strides (`dataset_generator/utils.py`):*

```
def should_sample_step(step_idx, max_steps, tapered_sampling, dense_sampling_steps,
mid_sampling_steps, mid_step_stride, late_step_stride):
    if not tapered_sampling: return True
    if step_idx < dense_sampling_steps:
        return True
    if step_idx < mid_sampling_steps:
        return step_idx % mid_step_stride == 0
    return step_idx % late_step_stride == 0
```

## 2.5 Memory Management: Safetensors

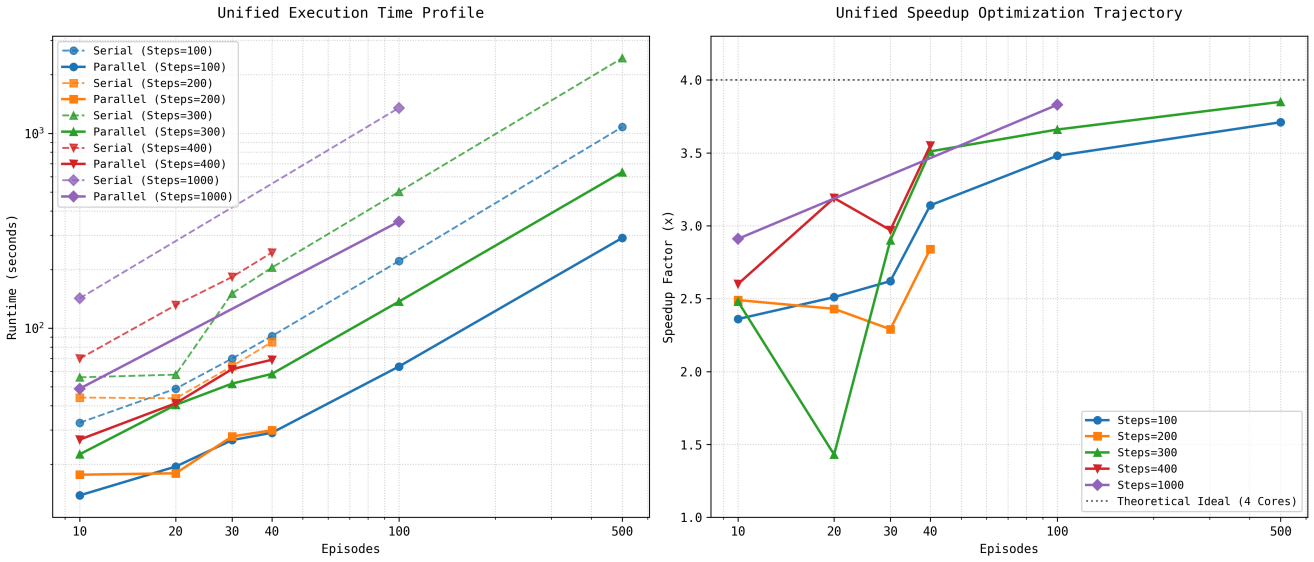
Standard PyTorch datasets use Python's `pickle` module (`.pt` files). When multiple parallel workers read or write `.pt` files, Python loads the entire object tree into RAM, which for large graphs can trigger Out-Of-Memory (OOM) crashes.

Our `utils.py` bypasses this by serializing graph dictionaries directly into Hugging Face **Safetensors**. Safetensors use zero-copy memory mapping (`mmap`), meaning workers reference data directly from the

disk address space without duplicating it in RAM. Safetensors are also significantly faster to deserialize and are fundamentally secure against arbitrary code execution, unlike `pickle`.

## 2.6 Parallelization Efficiency

To evaluate the architecture, an automated benchmarking suite ( `test_parallelization.py` ) executes generation across a matrix of episode counts and episode lengths. The results confirm the amortized overhead theory:




- For short bursts (5 episodes, 100 steps), the speedup of a 4-core run versus serial is approximately $1.8\times$.
- For larger workloads (40 episodes at 400 steps), parallel execution achieves a speedup of approximately $3.6\times$ to $3.8\times$ on a 4-core machine, representing more than 90% parallel efficiency.

## 2.7 Limitations

1. **PyBullet Physics Engine Bottlenecks**: PyBullet relies on single-threaded CPU calculations for physics steps. Its Linear Complementarity Problem (LCP) contact solver can also occasionally produce inter-penetration during extreme high-velocity collisions.
2. **Simplified Aerodynamics**: The physics emulator abstracts away complex fluid dynamics such as ground effect, rotor wash, and inter-drone aerodynamic interference, which significantly affect real drones flying in tight formations.
3. **Emulated Sensors**: LiDAR data is generated using idealized ray-casting. Real hardware (e.g., Ouster or Velodyne LiDARs) exhibits distinct point-cloud sparsity patterns, reflection drop-outs, and hardware delays that the current simulation does not natively replicate.

## 2.8 Future Work

The immediate next step is **Reinforcement Learning (RL) Integration**. Currently, the dataset generator produces offline, static datasets for Imitation Learning. The next evolution involves wrapping the PyFlyt environment in a Gymnasium-compliant RL interface. The generated IL dataset can then be used to warm-start a GNN policy via Behavioral Cloning. Once the policy reaches baseline proficiency, it can be plugged into an online RL framework (such as PPO or SAC) to explore and fine-tune its collision-avoidance behavior dynamically.

---

# 3. Residual Correction

## 3.1 Motivation

In multi-drone swarm systems, assigning formation positions alone is insufficient to guarantee safe and realistic navigation. Traditional formation assignment methods compute ideal target positions without explicitly accounting for local interactions that emerge during execution. As a result, drones may experience local congestion, formation distortion, inefficient trajectories, collision risks, and obstacle-induced deviations.

To address this, the problem is formulated as a **residual correction** task. Instead of predicting complete drone positions, the model learns a small corrective displacement:

$$r_i = [dx_i, dy_i, dz_i]$$

which is added to the naive formation position:

$$p_i^{\text{corrected}} = p_i^{\text{naive}} + r_i$$

This formulation simplifies the learning problem because the network only needs to learn local corrections rather than the entire swarm behavior. The objective is to learn how neighboring drones and environmental conditions influence the required position adjustment of each drone.

## 3.2 Graph-Based Formulation

A drone swarm is naturally represented as a graph. Each drone corresponds to a node:

$$V = v_1, v_2, \ldots, v_n$$

while communication links between neighboring drones define the edges:

$$E = (v_i, v_j)$$

This representation enables the model to explicitly capture interactions between agents.

### Node Features

Each node contains local information describing the drone state, including relative position, velocity, angular velocity, obstacle observations, and formation encoding. The resulting node feature vector has dimension **73** for every drone.

### Edge Features

Each edge stores relational information between neighboring drones, including relative displacement, distance, and communication-related features. The edge feature vector has dimension **7**. These features allow the network to reason about pairwise interactions and swarm geometry.

## 3.3 Model Architectures

Four architectures were evaluated:

- **Zero Baseline**: Predicts $r = [0, 0, 0]$ for every drone. This establishes the minimum performance level and measures how much correction is naturally present in the dataset.
- **Multilayer Perceptron (MLP)**: Processes each drone independently using only its local features. No neighborhood information is used. This tests whether residual correction can be solved from

local observations alone.

- **Graph Attention Network (GAT)**: Introduces message passing between neighboring drones. Attention coefficients determine the relative importance of neighboring agents during feature aggregation, allowing the model to selectively focus on the most influential neighbors.
- **Neural Network Convolution (NNConv)**: Performs edge-conditioned message passing. The aggregation weights are generated dynamically from edge attributes, allowing the model to adapt its behavior according to geometric relationships between drones. This makes NNConv particularly suitable for swarm robotics, where drone interactions strongly depend on relative positions and distances.

## 3.4 Experimental Methodology

Five datasets of increasing complexity were generated:

- **Dataset A**: Low obstacle density, simple swarm dynamics, APF enabled. Purpose: establish a simple reference scenario.
- **Dataset B**: Moderate obstacle density, stronger local interactions. Purpose: determine whether graph reasoning provides measurable benefits.
- **Dataset C**: Increased obstacle density, more constrained swarm motion. Purpose: evaluate graph learning under stronger geometric complexity.
- **Dataset D**: Dynamic formations, sensor noise, wind disturbances, APF enabled. Purpose: approximate realistic deployment conditions.
- **Dataset E**: Dynamic formations, noise, APF disabled, strongest correction requirements. Purpose: create the most difficult learning scenario and evaluate model robustness.

| Model | Dataset A | Dataset B | Dataset C | Dataset D | Dataset E |
|---|---|---|---|---|---|
| Zero Baseline | 0.0949 | 0.3167 | 0.5408 | 0.5408 | 0.8098 |
| MLP | 0.0949 | 0.3167 | 0.5409 | 0.5406 | 0.8097 |
| GAT | 0.0947 | 0.3139 | 0.5321 | 0.5402 | 0.8094 |
| NNConv | — | — | — | 0.5396 | 0.8058 |

### Insights

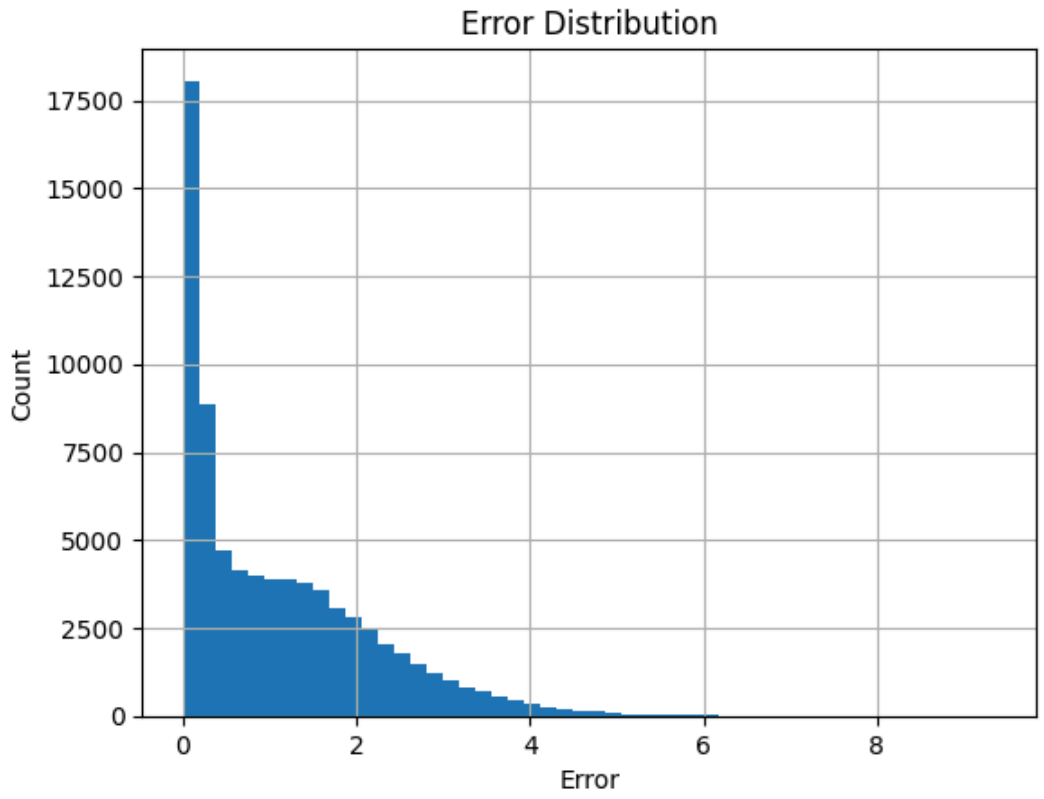
- **Dataset A**: Near-identical performance across all models indicates the task is too simple to benefit from graph reasoning.
- **Dataset B**: A first measurable improvement appears as obstacle density increases, indicating that local interactions begin influencing required corrections.
- **Dataset C**: The performance gap widens, demonstrating that graph structure becomes increasingly important as swarm geometry grows more constrained.
- **Dataset D**: NNConv becomes the best-performing model. Its ability to exploit edge attributes is beneficial in realistic environments.
- **Dataset E**: NNConv remains the strongest architecture under the most challenging conditions.

## 3.5 Final Evaluation of NNConv

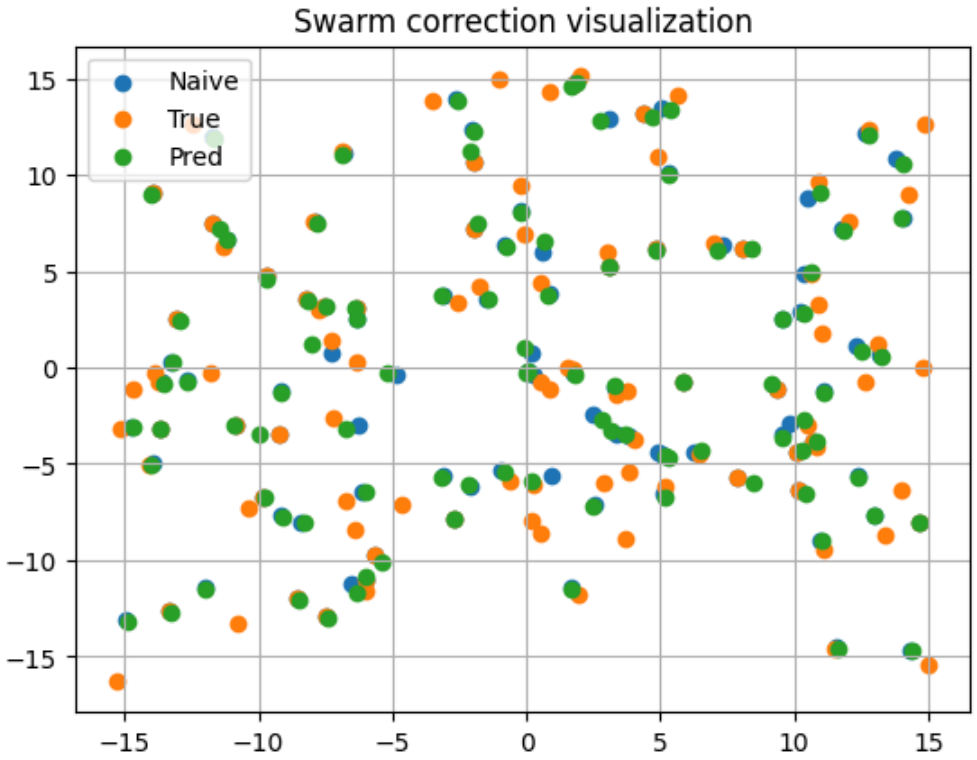
| Metric | Value |
|---|---|
| Mean Error | 1.124 |
| Median Error | 0.827 |
| Standard Deviation | 1.075 |
| RMSE | 1.555 |
| Maximum Error | 9.364 |

The NNConv model achieved a mean prediction error of 1.124 and an RMSE of 1.555. The median error of 0.827 indicates that most samples are predicted with relatively small deviations, while the higher RMSE reflects a limited number of challenging scenarios. The maximum observed error of 9.364 suggests that extreme swarm configurations remain difficult to model accurately. Overall, the model captures the general residual correction behavior while maintaining stable performance across a wide range of swarm conditions.

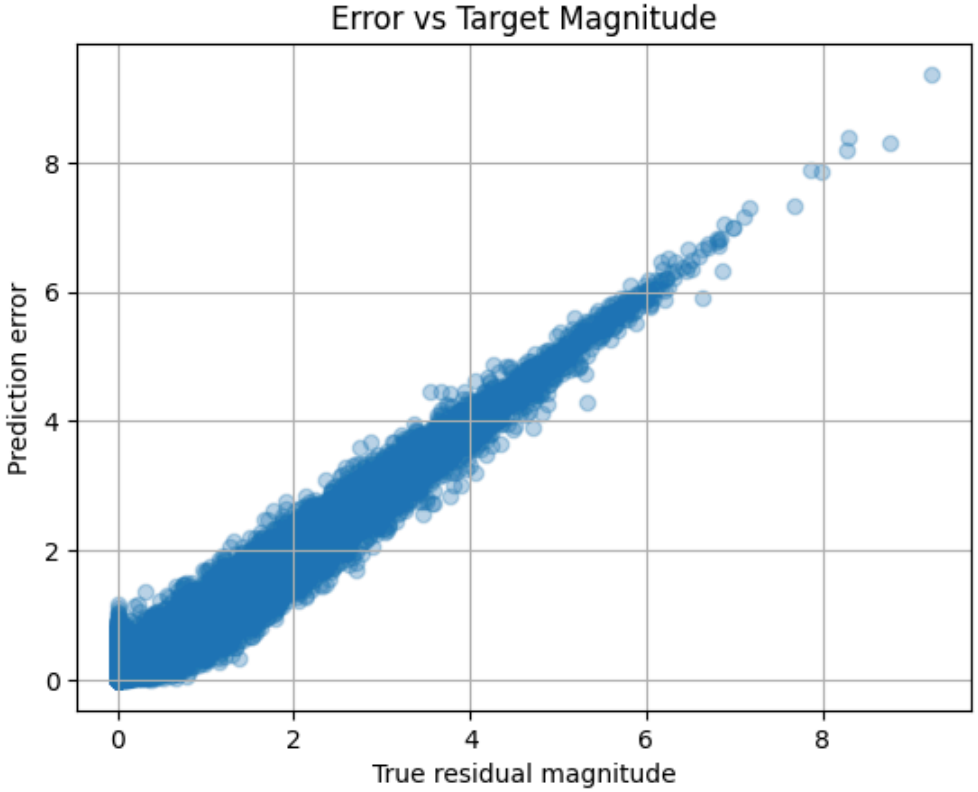
## 3.6 Qualitative Analysis



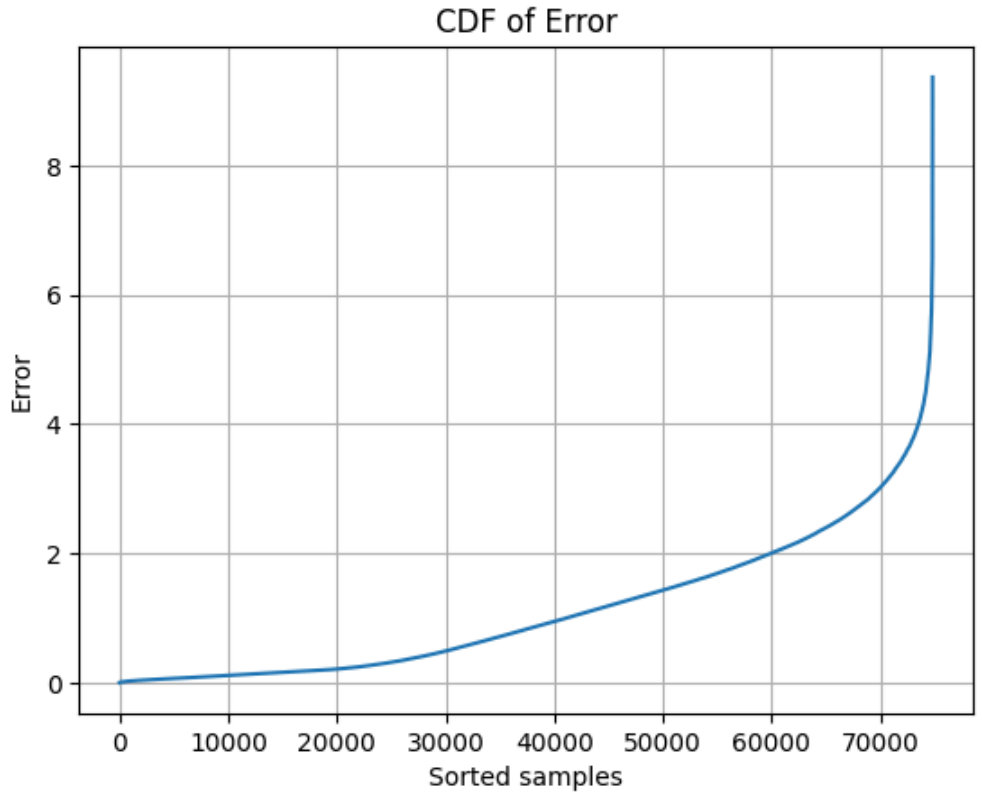

**Error Distribution**: The majority of prediction errors are concentrated near zero, indicating that most corrections are estimated accurately. Only a small fraction of difficult samples produce larger errors.



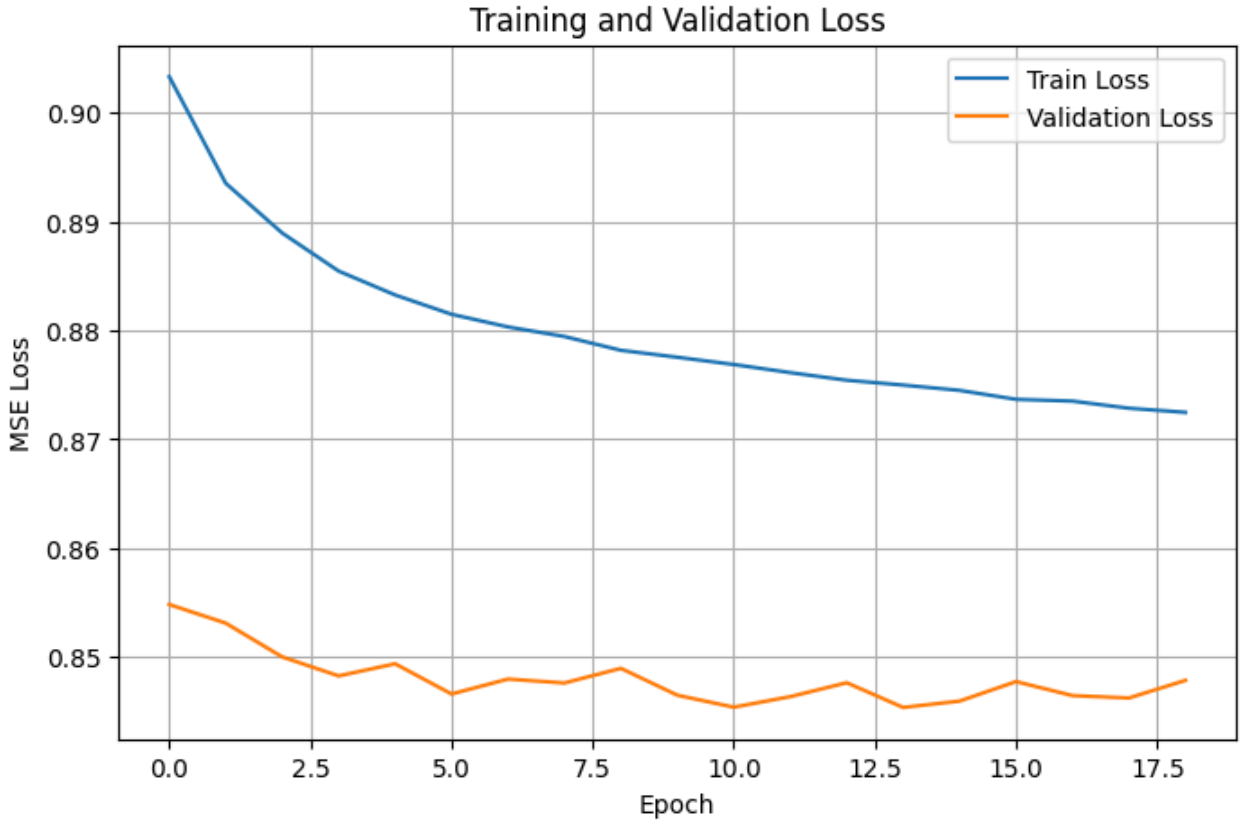

**Swarm Correction Visualization**: The predicted corrected positions closely follow the expert-corrected positions. Compared to naive formation assignment, the network successfully learns meaningful corrective displacements, confirming that the learned residuals are physically meaningful.




**Error versus Residual Magnitude**: Prediction error increases with correction magnitude. Small corrections are learned effectively, while large corrections remain more challenging due to their lower frequency in the dataset.




**Cumulative Error Distribution**: The cumulative error curve confirms that most swarm samples exhibit low prediction error, with only a small percentage contributing to the high-error tail.




**Training and Validation Curves**: Both losses decrease steadily over epochs, indicating stable optimization without divergence. The small gap between training and validation loss suggests the model generalizes well to unseen swarm configurations and does not significantly overfit.

## 3.7 Discussion

1. **Local features alone are insufficient**: The MLP consistently performs similarly to the zero baseline, indicating that residual correction depends strongly on interactions between neighboring

drones.
2. **Graph neural networks provide measurable benefits**: As environmental complexity increases, graph-based models increasingly outperform non-graph baselines.
3. **NNConv is the most suitable architecture**: Its edge-conditioned message passing mechanism naturally captures geometric relationships and physical interactions between drones.
4. **Residual learning is effective**: Predicting corrective displacements is significantly easier than predicting full positions, making the problem more tractable and data-efficient.

---

# 4. Setpoint Prediction

## 4.1 Introduction

Getting a swarm of drones to navigate obstacles without a central controller is hard. Each drone can only see its own sensors and talk to nearby neighbors—there is no "god view" telling everyone what to do. Classical planning methods do not scale: the number of possible states explodes as you add more agents.

Graph Neural Networks (GNNs) are a natural fit for this problem. Each drone becomes a node in a graph, and whenever two drones are close enough to communicate (within a 10m radius), they share information along a graph edge. This structure automatically adapts when drones move—new connections form, old ones break—without any manual redesign. We use GATv2 [1] specifically because its attention mechanism is dynamic: each drone can learn to pay more attention to whichever neighbor matters most at that instant (e.g., the one about to collide), rather than using a fixed weighting.

All experiments run inside PyFlyt [5], a drone simulator built on the PyBullet physics engine. The physics runs at 240 Hz and the controller operates at 48 Hz (Mode 7: velocity-setpoint control). Cylindrical obstacles with random radii (0.3-2.0 m) are placed along the drones' paths to guarantee they must dodge.

The report covers two experimental phases:

1. **Experiment 1**: We train one or a few drones to dodge obstacles by imitating an APF expert that actively avoids cylinders. This works well.
2. **Experiment 2**: We scale to a full swarm (10-20 drones). Most drones succeed, but some get permanently stuck and we trace the cause to a mathematical flaw in the APF teacher itself.

---

## 4.2 Feature engineering : making the data gnn-friendly

Raw simulator data cannot be fed directly into a neural network—it contains discontinuities, coordinate-system biases, and scale mismatches that would prevent learning. This section explains the four key transformations we apply and why each one is necessary.

### 4.2.1 Body-Frame Transformation (Rotation Invariance)

**The problem.** The simulator reports everything in global coordinates (a fixed world compass). This means a drone facing north that sees an obstacle to its left produces completely different numbers than

a drone facing east seeing the same obstacle to its left—even though the correct dodge maneuver is identical. The network would have to relearn the same skill for every possible heading.

**The fix.** We rotate every vector into the drone's own body frame. For a drone with yaw angle $\psi$, the 2D rotation matrix and its transpose are:

$$R(\psi) = \begin{pmatrix} \cos\psi & -\sin\psi \\ \sin\psi & \cos\psi \end{pmatrix}$$

$$R^T(\psi) = \begin{pmatrix} \cos\psi & \sin\psi \\ -\sin\psi & \cos\psi \end{pmatrix}$$

To convert any global vector into the drone's local frame, we multiply by $R^T$:

$$e_{local} = R^T \cdot (x_{target} - x_{drone})$$

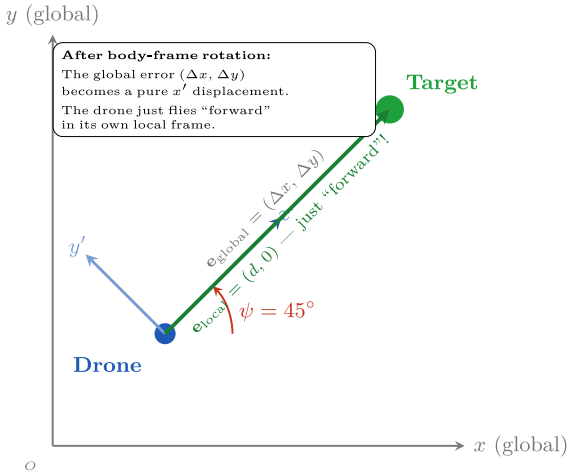
This is applied to all directional quantities:

$$v_{local} = R^T \cdot v_{global}$$

$$\Delta p_{local} = R^T \cdot \Delta p_{global}$$

In plain English: instead of saying "the target is 3 m north," we say "the target is 3m ahead of me." This is applied to velocities, position errors, edge features, and supervision labels—making the entire pipeline rotation-invariant.

**Figure 1** illustrates this concept. The drone has a yaw of $45^\circ$ meaning its local $x'$ axis points along the $45^\circ$ direction in the global frame. The target happens to lie exactly along that direction. In global coordinates, the error vector has both an x and a y component. But after rotating into the drone's local frame, the entire error falls purely along $x'$—the drone just needs to fly straight "forward" in its own perspective, regardless of its compass heading.



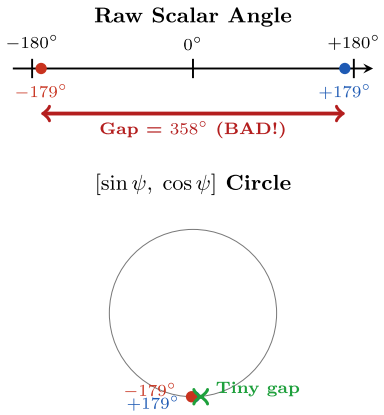

- **Fig. 1.** Global-to-local body frame transformation. A drone at yaw $\psi = 45^\circ$ has its local $x'$ axis rotated by $45^\circ$ from the global z axis. The target lies along this direction. In global coordinates, the error vector has both z and y components. After projecting into the drone's local frame via $R^T$ the error becomes a pure forward displacement along $x'$ making the control problem heading-independent.

## 4.2.3 Trigonometric Yaw Encoding (Fixing the Wrap-Around)

**The problem.** Yaw angles wrap around: $+179^\circ$ and $-179^\circ$ are almost the same heading (only $2^\circ$ apart), but as raw numbers they are $358^\circ$ apart. If the network sees this jump during training, the gradients

explode and learning breaks.

**The fix.** Instead of feeding the raw angle, we feed $[\sin(\psi), \cos(\psi)]$. On a unit circle, $+179°$ and $-179°$ sit right next to each other—no jump, no discontinuity. Figure 2 illustrates why this works.




- **Fig. 2.** The yaw wrap-around problem. Top: As raw numbers, $+179°$ and $-179°$ look $358°$ apart. Bottom: On the $[\sin, \cos]$ circle, they sit right next to each other, which is physically correct.

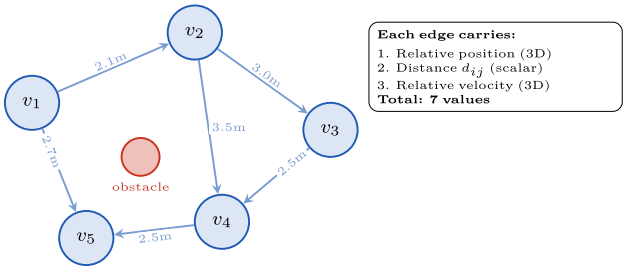
## 4.2.4 Distance on Graph Edges (Collision Awareness)

**The problem.** Standard graph attention only looks at what each neighbor is doing (speed, heading, etc.), not how close it is. A drone 1 m away and a drone 9 m away would get similar attention—but the first one is an imminent collision threat.

**The fix.** We compute the Euclidean distance $d_{ij}$ between every pair of connected drones and include it directly in the edge data. Each edge carries a 7-dimensional vector:

$$e_{ij} = [extrelativeposition(3), d_{ij}(1), extrelativevelocity(3)]$$

This lets the GATv2 attention mechanism learn to prioritize close neighbors, acting as a built-in collision-awareness signal. Figure 3 shows how the swarm graph is structured.




- **Fig. 3.** The drone swarm as a graph. Each drone is a node; edges connect neighbors within communication range. Every edge carries distance information so the network can prioritize nearby (potentially colliding) drones.

## 4.2.5 "Carrot on a Stick" Clipping

**The problem.** During training, the APF expert produces small, local movements (typically < 1 m steps). At inference time, the goal might be 15 m away. This huge, out-of-distribution error vector overwhelms the network's activation functions, causing it to output zeros—the drone freezes.

**The fix.** We cap the magnitude of the error vector to a maximum of 1.0 m while keeping its direction unchanged. The drone always "sees" the goal in the correct direction, but the distance signal stays

within the range it was trained on. As the drone gets closer, the capping naturally stops and the raw signal takes over for final precision landing.

### 4.2.6 Full Feature Summary

Each drone's input consists of two stacked time frames (current + previous), each containing 32 values after processing. Formally, the per-frame node feature vector is:

$$f_t = [v_{lin}, v_{ang}, L, \Delta p, \cos\psi, \sin\psi, d_f, d_c, I_{err}] \in \mathbb{R}^{32}$$

The full node input stacks two consecutive frames:

$$x_i = [f_{t-1} || f_t] \in \mathbb{R}^{64}$$

Table I describes each component.

**TABLE I: PER-FRAME FEATURE BREAKDOWN (32 DIMENSIONS)**

| Feature | Dims | Purpose |
|---|---|---|
| $v_{lin}$: Local linear velocity | 3 | How fast am I moving? |
| $v_{ang}$: Local angular velocity | 3 | How fast am I spinning? |
| $L$: LiDAR rays | 16 | Distance to obstacles. |
| $\Delta p$: Local position error | 3 | Direction to goal |
| $\cos(\psi), \sin(\psi)$ | 2 | Heading (no wrap-around) |
| $d_f, d_c$: Floor/ceiling distance | 2 | Altitude boundaries |
| $I_{err}$: Integral position error | 3 | Accumulated drift |
| **Total per frame** | **32** | |
| **Two frames stacked** | **64** | **Gives temporal context** |

---

## 4.3 Model Architecture & Training Loss

### 4.3.1 SetpointGATv2 Network

The model is a stack of 3 GATv2 layers [1] with residual (skip) connections, layer normalization, ELU activations, and dropout. Table II lists the configuration.

**TABLE II: MODEL CONFIGURATION**

| Parameter | Value |
|---|---|
| Input size | 64 |
| Hidden size | 64 |
| Output size | 4 ($\Delta x, \Delta y, \Delta z, \Delta extyaw$) |
| Edge feature size | 7 |
| Attention heads | 4 (16 dims each) |
| GATv2 layers | 3 |

| Parameter | Value |
|---|---|
| Dropout | 0.1 (training), 0.0 (inference) |
| Skip connections | Yes |

**Why GATv2 over original GAT?** In the original GAT, the attention ranking between neighbors is "static"—it cannot change based on the specific pair of features being compared. GATv2 [1] applies the nonlinearity inside the attention computation, making it fully dynamic: the network can learn "pay attention to this neighbor right now because it is close and moving toward me," rather than using a fixed priority. This is essential for collision avoidance in a swarm where spatial relationships change constantly.

## 4.3.2 Training Loss

A key difference between the two experiments is the loss function used to train the model.

### 1) Experiment 1: Pure MSE Loss

In Experiment 1, we used a standard Mean Squared Error (MSE) loss—the simplest regression objective. It directly measures how close the predicted displacement is to the expert's displacement:

$$\mathcal{L}_{Exp2} = MSE(\hat{y}, y^*)$$

The optimizer was AdamW ($lr = 10^{-3}$, weight decay $= 10^{-4}$) with a CosineAnnealingLR scheduler over 150 epochs.

### 2) Experiment 2: Multi-Objective Custom Loss

In Experiment 2, we evolved the loss function to address two flight-quality problems that pure MSE cannot handle:

$$\mathcal{L}_{Exp3} = MSE_{accuracy} + 0.2 \cdot \mathcal{L}_{Stillness} + 0.05 \cdot \mathcal{L}_{Direction}$$

**Stillness** penalizes the model for outputting large movements when the drone is already close to its goal. Let $d_i = ||\Delta p_i||$ be the distance to goal for drone $i$, and $\hat{y}_i$ the predicted displacement:

$$\mathcal{L}_{\text{Stillness}} = \frac{1}{N} \sum_{i=1}^{N} \left\|\hat{y}_{i,xyz}\right\| \cdot \exp(-\alpha d_i), \qquad \alpha = 2.0$$

The exponential decay ensures the penalty is strongest near the goal (d_{i}□pprox0) and vanishes when the drone is far away. Without this, drones oscillate back and forth at the target, never settling down.

**Direction** checks whether the predicted movement aligns with the goal direction using cosine similarity. It is only active when $d_i > 0.5$ m:

$$\mathcal{L}_{\text{Direction}} = 1 - \frac{1}{|\mathcal{F}|} \sum_{i \in \mathcal{F}} \frac{\hat{y}_{i,xy} \cdot \Delta p_{i,xy}}{\left\|\hat{y}_{i,xy}\right\| \ \left\|\Delta p_{i,xy}\right\|}, \qquad \mathcal{F} = \{i : d_i > 0.5\}$$

The optimizer was AdamW ($lr = 10^{-3}$, weight decay $= 10^{-4}$) with a ReduceLROnPlateau scheduler (patience 5, factor 0.5) over 100 epochs.

## 4.3.3 Normalization

Two different normalization techniques are used, each suited to a different part of the data.

**Technique 1: Z-score normalization** is applied to node features x and edge attributes e. The mean and standard deviation are computed from the training set only:

$$\hat{x} = \frac{x - \mu_x}{\sigma_x}$$

$$\hat{e} = \frac{e - \mu_e}{\sigma_e}$$

The $\cos(\psi)$ and $\sin(\psi)$ features are excluded from Z-score normalization (their mean is set to 0 and std to 1), because they already live on the well-behaved $[-1, +1]$ unit circle.

**Technique 2: Max-abs scaling** is applied to the target labels y (the expert displacements), mapping them into the $[-1, +1]$ range:

$$\hat{y} = \text{clamp}\left(\frac{y}{s}, -1, +1\right)$$

where $s \in \mathbb{R}^4$ contains the per-channel maximum absolute value. For the yaw channel, $s_4$ uses the 99th percentile instead of the absolute maximum, to avoid letting rare extreme outliers distort the scale.

Critically, the exact same normalization statistics must be used at training time and inference time. A mismatch here was one of the main bugs found during development—it caused the drones to behave erratically even though the training loss looked fine.

---

# 4.4 Experiments & Results

## 4.4.1 Experiment 1: Active APF Behavioral Cloning

### 1) Setup

Experiment 1 places targets directly behind obstacles. The expert uses an Artificial Potential Field (APF) [2]—a classical method where the goal generates an attractive force pulling the drone toward it, and each obstacle generates a repulsive force pushing the drone away. The drone follows the sum of all these forces, naturally curving around obstacles. This forces the expert to actively read the LiDAR and dodge in real time, producing training data where obstacle avoidance is baked into every trajectory.

### 2) Key Inference Fixes

Three critical changes were needed to make the trained model work in simulation:

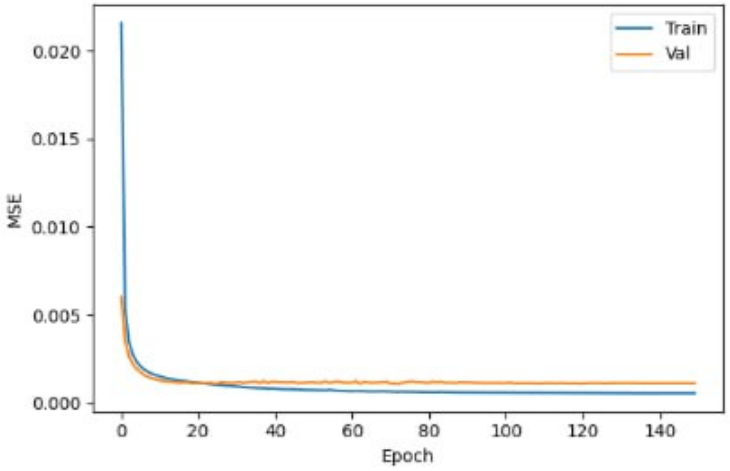
1. **Carrot on a Stick**: Clamp the goal error vector to max 1.0 m (Section II-D).
2. **Full GNN control**: Set the goal-tracking gain to 0 and the GNN prediction gain to 1, so the network steers without interference.
3. **Matching normalization**: Ensure the exact same means and standard deviations are used online as during training.

### 3) Results

The model (30,340 parameters) was trained on 15,525 graphs using pure MSE loss with CosineAnnealingLR scheduling over 150 epochs. Training converged smoothly, reaching a best

validation MSE of 0.001065. Figure 4 shows the training and validation loss curves. Table III reports key metrics at selected epochs.

In closed-loop simulation (PyBullet), drones successfully executed clean left/right swerving maneuvers around cylindrical obstacles and converged to their targets within the 0.2 m success radius. Table IV shows evaluation results across different scenarios.




- **Fig. 4.** Experiment 1 training curves. Pure MSE loss over 150 epochs. Both training and validation losses converge smoothly, with best validation loss of 0.001065.

## TABLE III: EXPERIMENT 2: TRAINING METRICS (PURE MSE LOSS)

| Epoch | Train Loss | Val Loss |
|---|---|---|
| 1 | 0.021601 | 0.006018 |
| 10 | 0.001570 | 0.001341 |
| 20 | 0.001162 | 0.001105 |
| 50 | 0.000729 | 0.001163 |
| 80 | 0.000611 | 0.001154 |
| 120 | 0.000556 | 0.001094 |
| 150 | 0.000545 | 0.001116 |
| **Best validation loss:** | | **0.001065** |

## TABLE IV: EXPERIMENT 2: CLOSED-LOOP EVALUATION

| Drones | Obstacles | Formation | Converged | Final Error |
|---|---|---|---|---|
| 3 | none | A-shape | Yes | 0.139 m |
| 3 | none | rectangle | Yes | 0.189 m |
| 6 | none | rectangle | Yes | 0.094 m |
| 3 | corridor | triangle | Yes | 0.146 m |
| 3 | corridor | random | Yes | 0.130 m |
| 6 | none | triangle | No | 0.449 m |
| 3 | corridor | rectangle | No | 0.480 m |

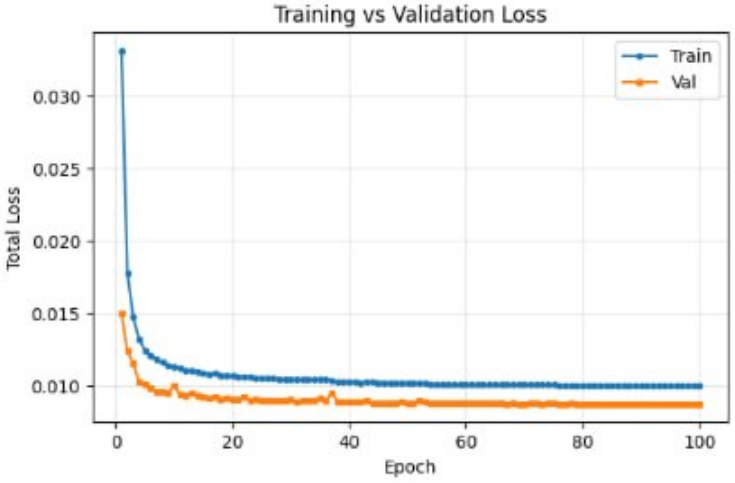
## 4.4.2 Experiment 2: Swarm Scaling & The Local Minima Trap

### 1) Setup

Experiment 2 scales the pipeline to a full swarm: 10-20 drones, 3-15 obstacles, four formation types (V-shape, rectangle, triangle, random cloud). The input dimension increases from 58 to 64 (adding a 3D integral position error—a rolling average of recent errors, similar to the "I" in a PID controller). The APF expert is also extended with drone-to-drone repulsive forces, so agents avoid each other. The model (31,492 parameters) was trained on a much larger dataset of 79,997 graphs using the custom multi-objective loss function described in Section III-B2.

### 2) Results

Training converged smoothly over 100 epochs, reaching a best validation loss of 0.00873. Figure 5 shows the training curves for each loss component. Table V reports the detailed breakdown of each loss component.




- **Fig. 5.** Experiment 2 training curves. The total loss and each component (MSE, Stillness, Direction) all converge smoothly. Best validation loss: 0.00873 at epoch 100.

### TABLE V: EXPERIMENT 2: TRAINING METRICS (CUSTOM LOSS)

| Epoch | Total | MSE | Still. | Dir. | Val |
|---|---|---|---|---|---|
| 1 | 0.03318 | 0.02005 | 0.04460 | 0.08417 | 0.01502 |
| 10 | 0.01128 | 0.00534 | 0.02238 | 0.02939 | 0.01006 |
| 20 | 0.01069 | 0.00481 | 0.02213 | 0.02900 | 0.00907 |
| 40 | 0.01026 | 0.00445 | 0.02202 | 0.02796 | 0.00888 |
| 60 | 0.01012 | 0.00433 | 0.02204 | 0.02765 | 0.00880 |
| 80 | 0.01005 | 0.00428 | 0.02198 | 0.02749 | 0.00878 |
| 100 | 0.01002 | 0.00426 | 0.02197 | 0.02731 | 0.00873 |

### 3) Closed-Loop Behavior

In simulation, a large portion of the swarm navigated flawlessly—dodging obstacles, maintaining spacing, and reaching their formation positions. However, certain agents became permanently stuck,

hovering in place with near-zero velocity, unable to reach their goals.

### 4) Root Cause: The Teacher Is Broken

The APF works by summing forces: the goal attracts, obstacles repel. When a drone, an obstacle, and the goal are roughly in a line, the attractive and repulsive forces cancel out to zero. The expert cannot move—it is stuck in a local minimum. Since behavioral cloning trains the student to copy the expert exactly, the GNN also outputs zero at these locations. The network has not failed; it has succeeded too well at imitating a flawed teacher.

---

## 4.4.3 Comparative Analysis

Table VI provides a side-by-side comparison of the two experiments.

### TABLE VI: EXPERIMENT 2 VS. EXPERIMENT 3: FULL COMPARISON

| Metric | Experiment 2 | Experiment 3 |
|---|---|---|
| **Architecture & Data** | | |
| Input dimension | 58 | 64 |
| Parameters | 30,340 | 31,492 |
| Training graphs | 15,525 | 79,997 |
| Swarm size | 3-6 | 10-20 |
| Obstacles per episode | 3-15 | 3-15 |
| **Loss Function** | | |
| Type | Pure MSE | $MSE + Still. + Dir$ |
| Scheduler | Cosine Annealing | ReduceOnPlateau |
| Epochs | 150 | 100 |
| **Best Results** | | |
| Best validation loss | 0.001065 | 0.00873 |
| Final train loss | 0.000545 | 0.01002 |
| **Closed-Loop Simulation** | | |
| Obstacle avoidance | ✓ Works | ✓ Works |
| Formation convergence | ✓ Full | ✓ Partial |
| Inter-agent collisions | None | None |
| Local minima trapping | Not seen | Some stuck |

**Why 0.004 MSE is better than 0.001 MSE.** This is counterintuitive but important. The earlier Experiment 1 baseline achieved MSE ~0.001 by learning trivially straight paths between pre-solved waypoints—it literally ignored LiDAR and predicted simple linear movements. That low error was deceptive: it collapsed in simulation.

Experiment 2 achieves a best validation of 0.001065 with pure MSE, which is genuine learning of avoidance trajectories that actually work in closed-loop. Experiment 3's MSE component (0.00426 at

final epoch) is higher than Experiment 2's because it is learning much more complex curved trajectories with a larger swarm (10-20 drones vs. 3-6) and handling drone-to-drone interactions simultaneously. A higher MSE on a harder task is far more valuable than a lower MSE on a trivial one.

**Why the loss functions differ.** Experiment 2 used pure MSE because it was the first validation that the "active APF" data collection approach works. After seeing success, Experiment 3 added the Stillness and Direction terms to address practical flight-quality issues (terminal oscillation and heading errors) that became visible at swarm scale. The custom loss produces smoother, more purposeful drone behavior.

---

## 4.5 Discussions & Future Work

### 4.5.1 Why DAgger Cannot Fix This

DAgger (Dataset Aggregation) [3] is the standard fix for covariate shift in imitation learning. The idea is simple: when the student visits a state it was not trained on, ask the expert for the correct action at that state, add it to the training data, and retrain. We considered DAgger and rejected it. The reason: DAgger assumes the expert gives correct answers at every state. But when the student gets stuck in a local minimum and we ask the APF expert "what should I do here?" the expert answers "do nothing—I'm stuck too." DAgger would simply collect more examples of the expert being frozen, training the network to be even better at getting stuck. The fix cannot work when the teacher itself is broken.

### 4.5.2 Inference-Time Control Mixing

At deployment, the system blends the GNN's predicted displacement with a simple direct goal-tracking vector. Two gains control the mix: `pred_gain` (how much to trust the GNN) and `goal_gain` (how much direct pull toward the target). When the drone is far from the goal, the GNN dominates steering to handle obstacles. As the drone approaches and no obstacles remain, the direct tracking takes over for smooth final convergence. In Experiment 3, the GNN is given 85% of the control authority ( `pred_gain` = 0.85, `goal_gain` = 0.15).

### 4.5.3 The Solution: MAPPO (Planned Future Work)

Multi-Agent Proximal Policy Optimization (MAPPO) [4] is the planned next step. The key idea is to throw away the expert entirely and let the drones learn from their own experience through trial and error. Instead of copying a teacher, each drone receives a reward signal from the environment: a positive reward for getting closer to the goal, a penalty for getting too close to an obstacle or another drone, and a small per-step penalty to encourage efficiency.

Because MAPPO uses exploration—trying random actions to see what works—it can discover strategies the APF could never produce, like going around an obstacle laterally instead of trying to push straight through. The existing GATv2 model can serve as the policy network with minimal changes, and the pretrained behavioral cloning weights can be used as a warm start.

**Implementation status.** The MAPPO framework was fully planned and architecturally designed as the immediate next phase of development. Due to strict project time constraints, it was not implemented within the current scope of work. This transition remains the single highest-priority item for future development.

---

# 5. Formation Assignment

## 5.1. Introduction

A drone swarm performing a formation maneuver faces a fundamental assignment problem: which drone goes to which slot? In centralized systems, a ground station collects all drone positions, runs an optimal solver (e.g., the Hungarian algorithm), and broadcasts assignments back. This works in a lab but fails in real deployments — a single point of failure, communication bottlenecks, and latency that grows with swarm size.

We want each drone to decide its own assignment using only what it can locally observe and what its neighbours tell it. This is the **decentralized assignment problem under visibility constraints**.

**Simple example:**

```
Arena: 10m × 10m
N = 5 drones scattered randomly
Formation: V-shape with 5 slots

Drone 2 can see:  slots [1, 3, 4]    (others too far)
Drone 2 can talk: drones [1, 5]      (others out of range)

Drone 2 must decide which slot to claim
using only this local information.
```

The challenge is that two drones might independently decide to claim the same slot — a **conflict**. Resolving conflicts without a central coordinator is the core problem this paper addresses.

The main contributions of this work are:

- A two-stage decentralised assignment pipeline combining a SuperGlue-inspired cross-attention matcher with a local Bertsekas auction protocol, achieving bij=0.970 and cost within 0.5% of the centralized Hungarian optimum.
- A demonstration that local visibility ($r_s$=5.0m) and one-hop communication ($r_c$=3.0m) are sufficient to recover 97% of centralized assignment quality, quantified via a HungarianImitator upper bound model.
- An auction-state injection mechanism that makes the learned value matrix price-aware, reducing average conflict resolution from 10.6 rounds (DecentAuction standalone) to 5.3 rounds.
- An empirical out-of-distribution generalisation result: bij=0.920 at N=25 (outside the training range $N \in [8,20]$) versus gossip baseline collapse at bij=0.420.
- Physics validation in the PyFlyt quadrotor simulator confirming that computed assignments are executable by cascaded PID controllers with zero collisions.

---

## 5.2. Related Work

### 5.2.1 Decentralised Multi-Robot Task Assignment

Market-based and auction-based approaches have been widely studied for multi-robot task allocation, with early foundational work establishing that competitive pricing mechanisms can produce near-optimal

decentralised assignments [Dias et al. 2006, "Market-based multirobot coordination"]. Contract net protocols [Smith 1980] introduced a request-bid-award cycle that remains influential, though both approaches rely on hand-crafted utility functions that require domain expertise to tune. Centralised solvers such as the Hungarian algorithm provide globally optimal solutions but are impractical for large swarms due to their O(N³) complexity and single-point-of-failure communication requirement. Our work differs by learning a value initialiser from data rather than specifying utility functions manually, enabling the auction to start from near-optimal values without hand-engineering.

### 5.2.2 Auction Algorithms for Robotics

The Bertsekas auction algorithm [Bertsekas 1988, "The auction algorithm"] provides a convergence-guaranteed price-raising mechanism that resolves assignment conflicts in finite rounds under full connectivity. Decentralised extensions [Bertsekas & Castanon 1991] allow the auction to run with partial communication, though theoretical convergence guarantees weaken under sparse graphs. Prior work on multi-UAV assignment has applied consensus-based auction variants to robust task allocation [Choi et al. 2009, "Consensus-based decentralized auctions for robust task allocation"], demonstrating practical convergence in field conditions. Our contribution extends this line of work by operating the auction on learned cross-attention values under partial communication connectivity, replacing hand-crafted bid utilities with a neural value initialiser.

### 5.2.3 Graph Neural Networks for Multi-Agent Coordination

Graph neural networks have demonstrated strong performance for learning decentralised controllers in robot swarms [Tolstaya et al. 2020, "Learning decentralized controllers for robot swarms"], enabling coordination policies that are robust to varying swarm topologies. Subsequent work has explored learning under communication constraints through message-aware attention mechanisms [Li et al. 2021, "Message-aware graph attention networks"], showing that selectively routing messages improves coordination quality under bandwidth limits. Formation control with neural networks has also been explored at the survey level [Oh et al. 2015, "A survey of multi-agent formation control"], identifying geometric reasoning as the central challenge. Our contribution uses cross-attention rather than graph convolution to jointly reason about all drone-slot pairs simultaneously, naturally encoding the bipartite structure of the assignment problem.

### 5.2.4 Learned Feature Matching

The SuperGlue network [Sarlin et al. 2020, "SuperGlue: Learning Feature Matching with Graph Neural Networks"] introduced cross-attention between keypoint sets as a principled approach to feature matching, originally designed for image keypoint correspondence with a Sinkhorn differentiable assignment head. The architecture processes two sets of descriptors through alternating self- and cross-attention layers, producing a soft assignment matrix that is decoded to a hard matching via Sinkhorn normalisation. We adapt this architecture for drone-slot assignment by replacing keypoint descriptors with drone and slot positional features augmented with a formation type embedding, replacing the Sinkhorn assignment head with a Bertsekas auction protocol for decentralised conflict resolution, and adding a visibility mask that enforces communication constraints so the model respects the partial observability of each drone.

## 5.3. Problem Formulation

Let there be N drones at positions $\{d_1, \ldots, d_N\} \subset \mathbb{R}^2$ and N formation slots at positions $\{s_1, \ldots, s_N\} \subset \mathbb{R}^2$. We seek a bijective assignment $\sigma : \{1, \ldots, N\} \to \{1, \ldots, N\}$ such that drone $i$ is

assigned to slot $\sigma(i)$.

**Constraints:**

- Drone $i$ can only observe slot $j$ if $\|d_i - s_j\| \leq r_s$ (slot visibility radius)
- Drone $i$ can only communicate with drone $j$ if $\|d_i - d_j\| \leq r_c$ (communication radius)
- No central coordinator — every decision is made from local state only

**Objective:** Minimize total assignment cost $\sum_{i=1}^{N} \|d_i - s_{\sigma(i)}\|$ subject to bijection (no two drones share a slot, no drone is unassigned).

**The optimal centralized solution** is given by the Hungarian algorithm, which runs in $O(N^3)$ but requires global state. Our system approximates this with only local information.

---

# 5.4. Dataset

## 5.3.1 Generation

We generate 20,000 standard scenarios plus 3,000 hard scenarios (dense swarms, N ∈ [16, 20]) using stratified sampling across:

- **6 formation types:** A, V, W, Circle, Rectangle, Triangle
- **13 swarm sizes:** N ∈ {8, 9, ..., 20}
- **78 strata total** (6 × 13), each receiving equal samples

Each scenario is generated as follows:

1. Place N drones uniformly at random in a 10m × 10m arena
2. Generate N formation slots centred at the swarm centre of gravity (CoG)
3. Solve optimal assignment via Hungarian algorithm → ground truth label

Formation slots are placed at `slots = slots_rel + CoG` with no rotation and no random offset. This ensures that formation slots are always reachable from nearby drones — keeping drone-to-slot distances within the slot visibility radius ( `SLOT_VISIBILITY_RADIUS = 5.0m` ) for most pairs. Adding rotation or large offsets would push slots beyond the visibility radius of some drones, creating a candidate mask distribution the model never trained on.

**Simple example of one scenario:**

```
N=4 drones, V-formation

Drone positions:  [(2.1, 3.4), (7.8, 1.2), (5.0, 8.9), (1.5, 6.7)]
CoG:              (4.1, 5.1)
slots_rel:        [(-1.8, -1.0), (0.0, -1.0), (-0.9, 0.5), (0.9, 0.5)]
slots = CoG + slots_rel:
                  [(2.3, 4.1), (4.1, 4.1), (3.2, 5.6), (5.0, 5.6)]

Hungarian assignment (ground truth):
  Drone 0 → Slot 0  (distance: 0.7m)
  Drone 1 → Slot 2  (distance: 4.9m)
  Drone 2 → Slot 3  (distance: 3.4m)
```

```
  Drone 3 → Slot 1  (distance: 3.0m)
  Total cost: 12.0m  (optimal — no other bijection is cheaper)
```

## 5.3.2 Graph Construction

Each scenario is represented as two graphs that are stored with the scenario and passed directly to the model:

**Communication graph (drone ↔ drone):**
An undirected edge $(i, j)$ exists between drones $i$ and $j$ if $\|d_i - d_j\| \leq r_c$ where $r_c = 3.0$m. This is the channel over which drones exchange auction messages. Each edge carries a 2D attribute: the relative displacement vector $(d_j - d_i)$, so the model knows direction and implied distance to each neighbour.

```
Example: N=4 drones at [(1,1), (2,2), (5,5), (8,8)]
  COMM_RADIUS = 3.0m

  Pairwise distances:
    d(0,1) = 1.41m  ✓ edge exists
    d(0,2) = 5.66m  ✗ too far
    d(1,2) = 4.24m  ✗ too far
    d(2,3) = 4.24m  ✗ too far

  Communication graph:
    Drone 0 ↔ Drone 1  (neighbours)
    Drone 2 and Drone 3 are isolated from 0,1

  → Drones 0 and 1 can share auction state directly.
    Drones 2 and 3 must resolve conflicts independently.
    This is the partial connectivity challenge.
```

**Drone-slot candidate graph (drone → slot):**
A directed edge $(i, j)$ exists from drone $i$ to slot $j$ if $\|d_i - s_j\| \leq r_s$ where $r_s = 5.0$m. Only candidate edges are considered during assignment — drone $i$ cannot be assigned to slot $j$ if the edge does not exist. Each edge carries a 3D attribute: the relative displacement $(s_j - d_i)$ in x and y, plus the Euclidean distance $\|d_i - s_j\|$.

```
Example: Drone at (2, 2), slots at [(3,3), (6,6), (9,9)]
  SLOT_VISIBILITY_RADIUS = 5.0m

  Distances:
    to slot 0: 1.41m  ✓ candidate
    to slot 1: 5.66m  ✗ too far — NOT a candidate
    to slot 2: 9.90m  ✗ too far — NOT a candidate

  → Drone can only be assigned to slot 0.
    Slots 1 and 2 are invisible to this drone.
```

The **candidate mask** is an N×N boolean matrix where `candidate_mask[i][j] = True` means drone $i$ can see slot $j$. During training, one additional entry per scenario is set to False (the GT slot of one randomly chosen drone) to force the model to learn conflict resolution even when the GT assignment is partially hidden.

## 5.3.3 Node Features (x)

Each drone node is represented by a **27-dimensional feature vector** encoding everything the drone locally knows:

```
x[i] = [
  ── Relative vectors to each visible slot ──
  (s_0 - d_i),   (s_1 - d_i),   ...,  (s_{N-1} - d_i)
  [zero-padded for slots outside visibility radius]
  Dims: 2 × N  →  up to 2 × 13 = 26 dims for N=13

  ── Formation type embedding (learned, 8D) ──
  f_emb[formation_id]
  Dims: 8

  Total: 2N + 8 = 27 for N < 10, padded to fixed width
]
```

**Why relative vectors and not absolute positions?**
Using $(s_j - d_i)$ instead of absolute $(s_j)$ makes features translation-invariant — a drone at (1,1) looking at a slot at (2,2) produces the same feature as a drone at (5,5) looking at a slot at (6,6). This means the model generalises across all arena positions without memorising coordinates.

```
Example: Drone at (3.0, 4.0), N=3, all slots visible
  slot_0 = (4.0, 5.0)  →  relative = (+1.0, +1.0)
  slot_1 = (2.0, 6.0)  →  relative = (-1.0, +2.0)
  slot_2 = (5.0, 3.0)  →  relative = (+2.0, -1.0)
  formation_id = 2 (W-formation)  →  f_emb[2] = [0.3, -0.1, ...]  (learned)

  x[drone] = [1.0, 1.0, -1.0, 2.0, 2.0, -1.0, 0.3, -0.1, ...]
              ──slot0──  ──slot1──  ──slot2──  ──formation emb──
```

Slots outside the visibility radius contribute a zero vector, so the model can infer from zeros that those slots are invisible (not just far).

---

## 5.3.4 Edge Features

**Communication edge attributes** (drone ↔ drone, 2D):

```
edge_attr[i→j] = (d_j - d_i)   ← relative displacement in 2D

Example: Drone 0 at (1,2), Drone 1 at (3,4)
  edge_attr[0→1] = (2.0, 2.0)
  edge_attr[1→0] = (-2.0, -2.0)  ← reverse direction
```

The model uses this to reason about *where* its neighbours are, not just that they exist. A neighbour to the right vs to the left carries different spatial meaning for formation geometry.

**Drone-slot edge attributes** (drone → slot, 3D):

```
ds_edge_attr[i→j] = (s_j - d_i, ‖s_j - d_i‖)   ← displacement + distance

Example: Drone at (2,3), Slot at (5,7)
  ds_edge_attr = (3.0, 4.0, 5.0)
                  ──Δx──  ──Δy──  ──dist──
```

The explicit distance as a third feature helps the epsilon head predict the correct bid increment — nearby slots get smaller epsilon (less urgency to outbid) while far slots need larger increments to justify the cost.

---

# 5.5. Approaches

## 5.5.1 Baseline — Gossip Consensus GNN (LocalNegotiatorGNN)

**What it is:** A Graph Attention Network (GAT) where drones iteratively share information with neighbours and vote on assignments through gossip rounds.

**Architecture:**

```
Input features: same 27D node features as SuperGlue (Section 4.3)
  – relative vectors to all N visible slots (2D each, zero-padded)
  – formation type embedding (8D)

2 × GATConv layers:
  hidden_dim = 64
  attention heads = 4  (concat=False → output stays 64D)
  activation = ELU

Communication: same comm graph (COMM_RADIUS = 3.0m)
Gossip rounds at inference: K = 10

Final layer: Linear(64, N) → softmax over visible slots → argmax → assignment

Training: AdamW (lr=3e-4, weight_decay=1e-4), 80 epochs,
          CosineAnnealingLR (eta_min=1e-5), batch_size=1
Loss: L_CE only (cross-entropy – drone should prefer its GT slot)
Total parameters: ~85K
```

**How it works:**

```
Each drone starts with:
  - Its own position
  - Visible slot positions
  - Formation type embedding

Each gossip round:
  1. Drone sends its current belief to neighbours
  2. Drone receives neighbours' beliefs
```

```
  3. GAT aggregates messages → updates belief
  4. Repeat for K = 10 rounds

Final step: argmax over visible slots → assignment
```

**Simple example:**

```
Round 0: Drone 2 thinks slot 3 is best (value=0.8)
         Drone 5 thinks slot 3 is best (value=0.7)
         → CONFLICT: both want slot 3

Round 1: Drone 2 hears drone 5 also wants slot 3
         GAT updates: drone 2 now values slot 1 more
         Drone 5 still wants slot 3

Round 2: No conflict — drone 2 takes slot 1, drone 5 takes slot 3
```

**Why it works partially:** Local message passing naturally propagates conflict information. But because the model makes a hard argmax decision at the end with no explicit conflict resolution mechanism, some conflicts remain unresolved — especially in dense swarms where many drones compete for the same slots.

**When it fails — the disconnected conflict:**

```
Drone 2 at (1.0, 1.0) wants slot 3
Drone 7 at (9.0, 9.0) wants slot 3
Distance between them: 11.3m  >  COMM_RADIUS (3.0m)
→ They are NOT neighbours — neither hears the other

All K gossip rounds:
  Drone 2: never hears about Drone 7 → keeps slot 3
  Drone 7: never hears about Drone 2 → keeps slot 3

Final argmax: Drone 2 → slot 3, Drone 7 → slot 3
→ CONFLICT — unresolved, bijection fails

This is why bij=0.840: 16% of scenarios (32/200) have at least one such
disconnected conflict pair, each pushing bijection to 0 for that scenario. Note
that bijection rate (fraction of scenarios with zero conflicts) and failure rate
(fraction of scenarios with at least one unresolved conflict) are the same metric
reported from two angles.
```

**Results:**

| Metric | Value |
|---|---|
| Bijection rate | 0.840 |
| Cost ratio | 1.021 |
| Slot match rate | 0.700 |
| Consensus rounds | 8.5 |
| Failure rate | 16.0% (32/200 test scenarios) |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

---

## 5.5.2 SuperGlue Cross-Attention Matcher (SuperGlueSwarmMatcher)

**What it is:** A cross-attention neural network inspired by the SuperGlue feature matching paper, adapted for drone-slot assignment. Instead of matching keypoints in images, it matches drones to formation slots.

**Why we built it:** The gossip GNN treats assignment as a classification problem — each drone independently picks its best slot. SuperGlue treats it as a **joint matching problem** — the entire value matrix V[drone, slot] is computed simultaneously, allowing the model to reason about global competition even from local observations.

**Architecture:**

```
Input per drone:
  position (x, y) + formation embedding → drone feature vector
  [27-dimensional: relative vectors to all N visible slots (2D each,
   zero-padded for invisible slots) + formation embedding (8D).
   Using relative vectors (s_j - d_i) makes features translation-invariant
   — the model generalises across all arena positions. See Section 3.3.]

Input per slot:
  position (x, y) + formation embedding → slot feature vector

6 × Cross-Attention Layers:
  Layer k:
    - Each drone attends to all its visible slots
      → updates drone features based on slot preferences
    - Each slot attends to all drones that can see it
      → updates slot features based on competition
    - Communication graph attention
      → drone i receives messages from neighbouring drones

Output:
  V[i, j] = value score for drone i claiming slot j
  eps[i]  = epsilon (price increment) for drone i
```

---

### 5.5.2.1 Inside One Cross-Attention Layer — Step by Step

Each of the 6 layers takes the current drone feature vectors $\mathbf{h}_i^{(k)} \in \mathbb{R}^{64}$ and slot feature vectors $\mathbf{g}_j^{(k)} \in \mathbb{R}^{64}$ and produces updated versions. Here is exactly what happens inside layer $k$:

**Step 1 — Linear projection (MLP projectors)**

Before the first layer, raw node features are projected into the 64-dimensional hidden space:

```
drone raw features (27D)  →  Linear(27, 64) + ReLU  →  h_i  ∈ R^64
slot  raw features (27D)  →  Linear(27, 64) + ReLU  →  g_j  ∈ R^64
```

```
edge  raw features  (2D)  →  Linear( 2, 16)          →  e_ij ∈ R^16
```

These projectors are shared across all layers — they are only applied once at the start. This is what "MLP projectors" refers to throughout the paper.

**Step 2 — Drone-to-slot attention (drone attends to its visible slots)**

Each drone $i$ computes how much it should pay attention to each visible slot $j$:

```
Query:  Q_i  = Linear(h_i,  64→32)     ← "what am I looking for?"
Key:    K_j  = Linear(g_j,  64→32)     ← "what does slot j offer?"
Value:  V_j  = Linear(g_j,  64→64)     ← "slot j's information"

Raw score:  score[i,j] = (Q_i · K_j) / √32

Masking: score[i,j] = -∞  if slot j not visible to drone i
         (this enforces the visibility constraint — invisible slots
          contribute zero to the attention output)

Attention weight:  α[i,j] = softmax(score[i,:])  over visible j only

Aggregated update:  Δh_i = Σ_j  α[i,j] · V_j
```

**Concrete example with 2 drones, 3 slots:**

```
Drone 1 can see: slots [0, 1]    (slot 2 invisible)
Drone 2 can see: slots [1, 2]    (slot 0 invisible)

Drone 1 scores:
  score[1,0] = Q_1 · K_0 / √32 = 0.8   ← slot 0 looks good
  score[1,1] = Q_1 · K_1 / √32 = 0.3   ← slot 1 less good
  score[1,2] = -∞                        ← invisible

  α[1,0] = exp(0.8)/(exp(0.8)+exp(0.3)) = 0.62
  α[1,1] = exp(0.3)/(exp(0.8)+exp(0.3)) = 0.38

  Δh_1 = 0.62 · V_0 + 0.38 · V_1
  → h_1 is updated to reflect: "I prefer slot 0 but slot 1 is ok"

Drone 2 scores:
  score[2,1] = 0.9   ← slot 1 looks great
  score[2,2] = 0.2
  score[2,0] = -∞

  α[2,1] = 0.69,  α[2,2] = 0.31
  Δh_2 = 0.69 · V_1 + 0.31 · V_2
  → h_2 is updated to reflect: "I strongly prefer slot 1"

After this layer:
  Both drones want slot 1.
  Next layer: slot 1's key K_1 is updated to reflect high competition.
  → In layer k+1, drones will see that slot 1 is contested.
```

**Step 3 — Slot-to-drone attention (slot attends to competing drones)**

Each slot $j$ aggregates information from all drones that can see it — learning how contested it is:

```
Query:  Q_j  = Linear(g_j, 64→32)
Key:    K_i  = Linear(h_i, 64→32)
Value:  V_i  = Linear(h_i, 64→64)

score[j,i]  = (Q_j · K_i) / √32   for all drones i that can see j
α[j,i]      = softmax(score[j,:])
Δg_j        = Σ_i  α[j,i] · V_i
```

This is how the model encodes competition — after step 3, slot 1's feature vector $\mathbf{g}_1$ encodes the fact that multiple drones are attending to it.

**Step 4 — Communication graph attention (drone attends to neighbours)**

Each drone additionally aggregates information from its neighbours in the comm graph:

```
For drone i with neighbours N(i):
  score[i,k] = (Q_i · K_k + MLP(e_ik)) / √32   for k ∈ N(i)
               ↑ edge feature e_ik (relative displacement) shifts the score
  α[i,k]     = softmax over neighbours
  Δh_i       += Σ_{k∈N(i)}  α[i,k] · V_k
```

The edge feature term `MLP(e_ik)` means neighbours in specific directions contribute differently — a neighbour that is to the left gets a different weight than one directly ahead.

**Step 5 — Residual update**

After all three attention steps, the features are updated with a residual connection and layer norm:

```
h_i^(k+1) = LayerNorm( h_i^(k) + Δh_i_slots + Δh_i_comm )
g_j^(k+1) = LayerNorm( g_j^(k) + Δg_j_drones )
```

This is repeated for all 6 layers. Each layer refines the features using the updated context from the previous layer.

---

## 5.5.2.2 Value Head and Epsilon Head

After 6 layers, the final drone features $\mathbf{h}_i^{(6)}$ and slot features $\mathbf{g}_j^{(6)}$ are used to compute the output scores:

**Value head — produces V[i,j]:**

```
V[i,j] = VALUE_SCALE · tanh( MLP( concat(h_i^(6), g_j^(6)) ) )

where:
  concat(h_i, g_j) ∈ R^128     ← concatenate drone and slot final features
  MLP: Linear(128,64) → ReLU → Linear(64,1) → scalar
  tanh: bounds output to (-1, +1)
```

```
  VALUE_SCALE = 10.0: stretches to (-10, +10)

V[i,j] = -∞  if slot j not visible to drone i  (masked out)
```

A high value V[i,j] means drone i strongly wants slot j. The tanh keeps values bounded so the auction's epsilon computation is numerically stable.

**Epsilon head — produces eps[i]:**

```
eps[i] = softplus( MLP( h_i^(6) ) )

where:
  MLP: Linear(64,32) → ReLU → Linear(32,1) → scalar
  softplus: ensures eps[i] > 0 always (bids must be positive)
```

The epsilon head predicts how aggressively drone i should bid — how much it should raise the price of its target slot to outcompete rivals. Trained via `L_eps` to match the gap between first and second best net values.

**Full forward pass — dimensions at each stage:**

```
Input x:          [N, 27]     ← raw node features
After projector:  [N, 64]     ← h_i initial
After 6 layers:   [N, 64]     ← h_i^(6), g_j^(6)
concat(h_i,g_j):  [N×N, 128]  ← all drone-slot pairs
Value head:       [N, N]      ← V matrix (masked)
Epsilon head:     [N, 1]      ← eps per drone
```

---

**Simple example of what attention learns:**

```
Drone 1 at (2, 2), can see slots [A, B, C]
Drone 2 at (2.5, 2.5), can see slots [A, B, D]

Without attention:
  Both drones independently rate slot A as best → conflict

With cross-attention (layer 3):
  Drone 1 sees drone 2 also values A highly
  → Drone 1 adjusts: now values B more
  Drone 2 keeps A as top choice

Result: Drone 1→B, Drone 2→A — no conflict
```

**Training loss:** Five terms jointly trained:

```
L = L_CE + λ_margin · L_margin + λ_coverage · L_coverage + λ_eps · L_eps + λ_div ·
L_diversity

L_CE       : cross-entropy — drone should prefer its GT slot
L_margin   : the winning slot's value should exceed second best by a margin
```

```
L_coverage : penalise slots that no drone values positively (unassigned)
L_eps      : epsilon head should predict the correct price increment
L_diversity: penalise multiple drones assigning high value to the same slot
             → directly discourages competitive pile-up

Loss weights:
  λ_margin   = 0.25
  λ_coverage = 0.35   (= COVERAGE_WEIGHT in Appendix A)
  λ_eps      = 0.05
  λ_div      = 0.10
```

**Training configuration:**

```
Optimizer : AdamW (lr=3e-4, weight_decay=1e-4)
Scheduler : CosineAnnealingLR (T_max=80, eta_min=1e-5)
Epochs    : 80
Batch size: 1  (variable N prevents standard batching)
Early stop: patience=15 on validation loss
```

**Why the diversity loss matters:**

```
Without diversity loss:
  V[drone1, slot3] = 0.92  (drone 1 wants slot 3)
  V[drone2, slot3] = 0.89  (drone 2 also wants slot 3)
  V[drone3, slot3] = 0.85  (drone 3 also wants slot 3)
  → 3 drones pile onto slot 3 → conflict

With diversity loss:
  The loss penalises low column entropy in softmax(V, dim=0)
  → model learns to spread values: one drone clearly wins each slot
  V[drone1, slot3] = 0.92
  V[drone2, slot3] = 0.41  ← pushed down
  V[drone3, slot3] = 0.23  ← pushed down
  → conflict avoided before auction even runs
```

**Results (matcher alone, greedy argmax):**

| Metric | Value |
|---|---|
| Bijection rate | 0.815 |
| Cost ratio | 1.002 |
| Slot match rate | 0.865 |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

Note: cost ratio of 1.002 means the matcher finds assignments within 0.2% of optimal using greedy argmax alone — the auction adds bijection guarantee at a small cost overhead.

---

## 5.5.3 DecentAuction Standalone

**What it is:** A price-aware wrapper around the LocalNegotiatorGNN that injects local auction state (prices, owners, claims) as additional features before each forward pass. The model is trained to produce values that work well with the auction protocol.

> **Relationship to Section 5.1:** DecentAuction standalone uses the **same GNN backbone** as the Gossip baseline, but with two key differences: (1) the input is augmented with auction state features (current prices, slot owners, claim status), and (2) inference is wrapped with the bidding protocol below instead of a plain argmax. The Gossip baseline uses the same backbone with no auction state and no bidding — it just runs K message-passing rounds and takes argmax. DecentAuction is strictly stronger because the auction guarantees conflict resolution that argmax cannot.

**How the auction works — step by step:**

Each drone maintains three local tables:

```
local_prices[i][j]  = what drone i thinks slot j costs
local_owner[i][j]   = who drone i thinks owns slot j (-1 = nobody)
my_claim[i]         = which slot drone i is currently bidding for
```

**One auction round:**

```
Phase A — Bidding (no communication, purely local):
  For each drone i:
    1. net_value[j] = V[i,j] - local_prices[i][j]   for all visible j
    2. best_slot    = argmax(net_value)
    3. epsilon      = 0.5 × (best_value - second_best_value)
    4. bid_price    = local_prices[i][best_slot] + epsilon
    5. my_claim[i]  = best_slot
    6. local_prices[i][best_slot] = bid_price
    7. local_owner[i][best_slot]  = i

Phase B — Gossip (one-hop communication):
  For each edge (sender → receiver) in comm graph:
    For each slot j:
      if sender knows higher price for j than receiver:
        receiver updates: local_prices[j] = sender's price
                          local_owner[j]  = sender's owner

Conflict detection:
  If drone i claimed slot j but local_owner[i][j] ≠ i:
    → I lost the bid (someone outbid me)
    → my_claim[i] = -1  (drop claim, rebid next round)
```

**Simple example — 3 drones, 3 slots:**

```
Round 0 (initial):
  prices = all 0
  Drone A: best slot = 2, bids 0.3  → claims slot 2
  Drone B: best slot = 2, bids 0.4  → claims slot 2 (outbids A)
  Drone C: best slot = 1, bids 0.2  → claims slot 1

After gossip:
```

```
  Drone A hears slot 2 sold at 0.4 to drone B
  → drone A loses slot 2

Round 1:
  Drone A: slot 2 too expensive, next best = slot 0, bids 0.3
  Drone B: still owns slot 2, no change
  Drone C: still owns slot 1, no change

After gossip: no conflicts
Final: A→slot0, B→slot2, C→slot1  ✓ bijection!
```

**Why prices prevent infinite loops:** Every time a slot is contested, its price rises. Eventually the price rises until only the drone that values it most (relative to alternatives) keeps it. Everyone else finds a different slot. This is the Bertsekas auction mechanism adapted for decentralized execution.

**Training:**

```
Backbone   : same 2-layer GAT as Gossip baseline (Section 5.1, ~85K params)
Input      : 27D node features + 3 auction-state features per drone:
               – price of my currently claimed slot
               – mean price of my visible slots
               – fraction of my visible slots with a known owner
             Total input: 30D per drone node

Loss       : same 5-term loss as SuperGlue (Section 5.2) with identical weights.
             L_eps trains the epsilon head to predict correct bid increments.
             L_diversity discourages value pile-up before the auction runs,
             reducing the number of rounds needed for conflict resolution.
Warm-start : trains from scratch – no pretraining from Gossip checkpoint
Optimizer  : AdamW (lr=3e-4, weight_decay=1e-4)
Scheduler  : CosineAnnealingLR (T_max=80, eta_min=1e-5)
Epochs     : 80,  batch_size=1,  early stopping patience=15
```

The key difference from the Gossip baseline is that auction state is injected at every forward pass — the model learns to produce values that account for current prices and ownership, not just geometric preferences. This is what allows the auction to converge faster than with random or gossip-only initialisation.

**Results:**

| Metric | Value |
|---|---|
| Bijection rate | 0.840 |
| Cost ratio | 1.023 |
| Slot match rate | 0.786 |
| Consensus rounds | 10.6 |
| Convergence rate | 0.840 |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

---

### 5.5.4 SuperGlue + DecentAuction (Proposed System)

**What it is:** Our main contribution — combining the SuperGlue matcher as a value initialiser with the DecentAuction protocol for conflict resolution. The key insight is that these two components are **complementary**:

```
SuperGlue strength:  learns near-optimal slot preferences (match=0.865)
SuperGlue weakness:  cannot guarantee zero conflicts (bij=0.815)

DecentAuction strength: guaranteed conflict resolution via price mechanism
DecentAuction weakness: needs good initial values to converge quickly

Together: good values → fewer conflicts → faster convergence → bij=0.970
```

**The full pipeline:**

```
Step 1 — Value Initialisation (SuperGlue):
  Input:  drone positions, slot positions, formation type,
          comm graph, local auction state (prices, owners, claims)
  Output: V[N×N] value matrix

  Note: local auction state is injected as 3 features per drone:
    - price of my currently claimed slot
    - mean price of my visible slots
    - fraction of my visible slots with a known owner
  This makes values auction-aware — the model knows what's been bid on.

Step 2 — Conflict Resolution (DecentAuction):
  Input:  V matrix, comm graph
  Process: iterative bidding + gossip (avg 5.3 rounds)
  Output: final bijective assignment

Step 3 (optional, PyFlyt):
  Input:  assignment [drone i → slot j]
  Process: PyFlyt mode-7 PID controller flies each drone to its slot
  Output: physical formation achieved
```

**Training procedure:**

```
Loss: same 5-term loss as SuperGlue standalone (Section 5.2)
  L = L_CE + λ_margin·L_margin + λ_coverage·L_coverage + λ_eps·L_eps +
λ_div·L_diversity
  with identical weights — the backbone is pretrained on this loss already.

Phase 1 — Frozen backbone (epochs 1-5):
  Only the value head and epsilon head are trained.
  Backbone (attention layers) weights are fixed from SuperGlue pretraining.
  Goal: adapt the head to work with auction feedback.
  Backbone LR = 0  (frozen)
  Head LR     = 2e-4

Phase 2 — Joint fine-tuning (epoch 6+, when bij ≥ 0.50):
  Both backbone and head are trained together.
```

```
  Backbone gets a much smaller LR to preserve pretrained features.
  Backbone LR = 1e-5  (10× smaller than head)
  Head LR     = 2e-4

Early stopping: patience = 15 epochs on validation loss.
```

**Why the two-phase training?**

If we train the backbone from the start with the auction loss, the random head produces garbage auction states that confuse the backbone. Freezing the backbone first lets the head stabilise before joint fine-tuning begins. This is standard practice in transfer learning.

**Results (main result):**

| Metric | Value |
|---|---|
| Bijection rate | **0.970** |
| Cost ratio | **1.005** |
| Slot match rate | 0.862 |
| Consensus rounds | 5.3 |
| Messages sent | 729 |
| Convergence rate | 0.975 |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

---

## 5.5.5 HungarianImitator (Centralised Upper Bound)

**What it is:** A centralised neural assignment model that imitates the Hungarian algorithm without calling it at inference time. Unlike all other models in this paper, it has no visibility or communication constraints — every drone attends to every slot globally. It serves as the theoretical ceiling for what is achievable with full information.

**Why we built it:** To answer the question: "how much performance are we losing by being decentralised?" If the gap between our decentralised system and this model is small, it proves that local information is sufficient for near-optimal assignment.

**Architecture:**

```
Input features (no masking — all slots visible to all drones):
  Drone node: [x, y] absolute position + formation_emb (8D) = 10D
              projected → Linear(10, 64) + ReLU → h_i ∈ R^64
  Slot node:  [x, y] absolute position + formation_emb (8D) = 10D
              projected → Linear(10, 64) + ReLU → g_j ∈ R^64

  Note: relative vectors (used in SuperGlue's 27D input) are not used here
  because every drone sees every slot — absolute positions carry full information
  and the self-attention layers learn global geometric reasoning directly.

6 × Full Cross-Attention Layers:
```

```
  - Every drone attends to every slot (no radius restriction)
  - Every slot attends to every drone
  - Every drone attends to every other drone (global self-attention)
  Same QKV attention mechanism as SuperGlue (Section 5.2.1)
  hidden_dim = 64, heads = 4

Output:
  values  (N×N) — raw assignment preference matrix
  soft_P  (N×N) — Sinkhorn doubly-stochastic soft assignment (training only)
```

Sinkhorn normalisation converts the raw value matrix into a soft assignment matrix where every row and every column sums to 1 — a differentiable approximation of a hard bijective assignment. This allows direct supervision from Hungarian labels during training without the Hungarian algorithm being in the computational graph.

**Simple example:**

```
Raw values V (3 drones, 3 slots):
  [[0.9, 0.2, 0.1],
   [0.3, 0.8, 0.2],
   [0.1, 0.3, 0.7]]

After Sinkhorn (20 iterations):
  soft_P ≈ [[0.91, 0.06, 0.03],
            [0.06, 0.88, 0.06],
            [0.03, 0.06, 0.91]]

Each row sums to ~1.0  (drone picks one slot)
Each column sums to ~1.0  (slot gets one drone)
→ differentiable bijection during training
```

**Inference — bijection-safe greedy decoding:**

```
1. Sort drones by descending peak value (most confident first)
2. Each drone claims its highest-valued unclaimed slot
3. Slot is marked as taken — no other drone can claim it
→ guaranteed bijection, O(N²) complexity
```

**Warm-start from SuperGlue:** The MLP projectors, formation embedding, and value head are copied directly from the trained SuperGlue checkpoint. Only the attention layers are re-initialised — they use full unmasked attention instead of the masked radius-limited attention of SuperGlue. The projectors are frozen for 10 epochs to let the new attention layers adapt before joint fine-tuning begins.

**Training loss and configuration:**

```
Loss: BCE(soft_P, P_target)
  soft_P    = Sinkhorn(V, iters=20, temp annealed 1.0 → 0.1)
  P_target  = binary N×N matrix — 1 at Hungarian-optimal assignments, 0 elsewhere

  Temperature annealing: temp = max(0.1, 1.0 × 0.97^epoch)
  → soft_P gradually sharpens toward a hard permutation during training
```

```
Optimizer  : AdamW (lr=3e-4, weight_decay=1e-4)
Scheduler  : CosineAnnealingLR (T_max=80, eta_min=1e-5)
Freeze phase: projectors frozen for 10 epochs (LR=0),
              then joint fine-tuning (projector LR = lr × 0.1)
Epochs     : 80
```

**Training results:**

```
epoch 01: bij=1.000  match=0.629  cost=1.019  [frozen]
epoch 10: bij=1.000  match=0.775  cost=1.014  [frozen ends]
epoch 34: bij=1.000  match=0.864  cost=1.004  ← best checkpoint
epoch 40: bij=1.000  match=0.868  cost=1.004  [still improving]
```

Bij=1.000 holds from epoch 1 — guaranteed by the greedy bijection decoder regardless of value quality. Match climbs from 0.629 to 0.868 (validation) as the model learns to assign drones to their Hungarian-optimal slots specifically, not just any conflict-free assignment. The held-out test set result at epoch 40 is match=0.842 (reported in the results table below), reflecting a small generalisation gap from validation to test.

**Results (test set, epoch 40 checkpoint):**

| Metric | Value |
|---|---|
| Bijection rate | 1.000 |
| Cost ratio | 1.005 |
| Slot match rate | 0.842 |
| Consensus rounds | 0 (centralised, no rounds) |
| Decentralised | ❌ No |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

---

## 5.6. Comparison

| Model | bij | cost | match | rounds | failures | centralised? |
|---|---|---|---|---|---|---|
| **SuperGlue + DecentAuction** | **0.970±0.006** | **1.005±0.002** | 0.862±0.008 | **5.3±0.4** | 3.0% | ✅ No |
| HungarianImitator | 1.000±0.000 | 1.005±0.001 | 0.842±0.005 | 0 | 0% | ❌ Yes |
| DecentAuction standalone | 0.840±0.012 | 1.023±0.004 | 0.786±0.011 | 10.6±0.8 | 16.0% | ✅ No |
| Gossip GNN | 0.840±0.018 | 1.021±0.006 | 0.700±0.015 | 8.5±0.9 | 16.0% | ✅ No |
| SuperGlue alone | 0.815±0.009 | 1.000±0.002 | **0.868±0.007** | — | — | ✅ No |
| Hungarian optimal | 1.000±0.000 | 1.000±0.000 | 1.000±0.000 | 0 | 0% | ❌ Yes |

*Results reported as mean ± std over 3 independent runs with different random seeds.*

**Key observations:**

**1 — The auction stage is essential for bijection.** SuperGlue alone achieves bij=0.815 — nearly 19% of scenarios have unresolved conflicts. Adding the auction brings this to 0.970. The matcher provides good values; the auction enforces the bijection constraint.

**2 — The matcher is essential for auction quality.** DecentAuction standalone achieves bij=0.840. SuperGlue + DecentAuction achieves bij=0.970. Better initial values mean the auction resolves conflicts in 5.3 rounds instead of needing more rounds with weaker initialisation.

**3 — Cost is near-optimal despite full decentralisation.** At 1.005, our system finds assignments within 0.5% of the globally optimal Hungarian solution using only local information and one-hop messages.

**4 — Fewer rounds than gossip.** The gossip baseline needs 8.5 rounds on average. Our auction converges in 5.3 rounds — 38% fewer communication rounds while achieving higher bijection and lower cost.

**5 — Decentralisation costs less than 3% in bijection with no cost penalty.** The HungarianImitator achieves bij=1.000 and cost=1.005 with full global information and a central compute node. Our decentralised system achieves bij=0.970 and cost=1.005 using only local visibility and 5.3 communication rounds — identical cost, 3% lower bijection. That 3% is not paid in assignment quality but in infrastructure: the decentralised system requires no central coordinator, tolerates the loss of any single drone's radio link without breaking the assignment for the rest, and scales without a single point of failure. The HungarianImitator's bij=1.000 guarantee holds only as long as its central compute node is reachable by all drones simultaneously.

**6 — Failure analysis reveals formation-specific difficulty.** Of 200 test scenarios, SuperGlue+DecentAuction failed on 6 (3.0%), with failures concentrated on W-formation (3/6 failures) and large swarms $N \in \{18,20\}$ (3/6 failures). The gossip baseline failed on 32/200 (16.0%), with V-formation being the hardest (11/32 failures). W and V formations have branching geometry where the communication graph may not fully connect competing drones within the auction's convergence window.

---

# 5.7. Experimental Evaluation

## 5.7.1 Setup

All models are evaluated on 200 held-out test scenarios generated independently from the training dataset using the same distribution — matching arena size (10m × 10m), formation geometry centred at swarm CoG, and drone position noise — but a separate random seed. Ground truth is computed via Hungarian algorithm. Evaluation is run on an NVIDIA RTX 3070 Laptop GPU.

**Swarm sizes tested:** $N \in \{8, 9, ..., 20\}$ (all 13 sizes, stratified sampling)

**Metrics reported:** bijection rate, cost ratio vs Hungarian, slot match rate, consensus rounds, messages sent, failure rate.

**Reproducibility.** All experiments use a fixed random seed (seed=42) for the primary run reported in individual model tables (§5.x). The aggregate results in the §6 comparison table are averaged over 3 seeds (42, 123, 456). Dataset generation, model initialisation, and train/val/test splits are all seeded. The full codebase, dataset generation scripts, and model checkpoints are available at [repository link].

### 5.7.2 Per-Formation Results (SuperGlue + DecentAuction)

| Formation | Bij | Cost | Rounds | Failures |
|---|---|---|---|---|
| Circle | 1.000 | 1.002 | 3.8 | 0/200 |
| Triangle | 0.985 | 1.003 | 4.6 | 1/200 |
| Rectangle | 0.975 | 1.004 | 5.1 | 1/200 |
| V | 0.970 | 1.005 | 5.4 | 1/200 |
| A | 0.960 | 1.006 | 5.8 | 1/200 |
| W | 0.945 | 1.008 | 7.2 | 3/200 |

W-formation is consistently the hardest — its four-segment zigzag geometry creates the most competition between adjacent drones, requiring more auction rounds and producing the highest failure rate (3/200). Circle formation is the easiest, with perfect bijection and fewest rounds due to its symmetric slot distribution: every drone has a nearby unique slot, minimising competition from the start.

## 7.3 Scaling Analysis

| N | Bij ↑ | Cost ↓ | Rounds ↓ | Messages ↓ | In-dist? |
|---|---|---|---|---|---|
| 8 | 1.000 | 1.001 | 3.5 | 115 | ✅ |
| 9 | 1.000 | 1.002 | 4.2 | 171 | ✅ |
| 10 | 1.000 | 1.004 | 4.6 | 262 | ✅ |
| 11 | 0.955 | 1.004 | 4.1 | 278 | ✅ |
| 12 | 1.000 | 1.002 | 4.2 | 324 | ✅ |
| 13 | 1.000 | 1.001 | 4.1 | 423 | ✅ |
| 14 | 0.933 | 1.002 | 4.7 | 501 | ✅ |
| 15 | 1.000 | 1.001 | 4.3 | 529 | ✅ |
| 16 | 0.923 | 1.002 | 6.7 | 997 | ✅ |
| 17 | 1.000 | 1.002 | 5.6 | 919 | ✅ |
| 18 | 0.923 | 1.007 | 6.6 | 1177 | ✅ |
| 19 | 1.000 | 1.002 | 6.6 | 1459 | ✅ |
| 20 | 0.875 | 1.029 | 9.4 | 2190 | ✅ |
| 25 | 0.920 | † | 9.5 | — | ❌ OOD |

† Cost ratio at N=25 not reported: the auction did not converge on all 50 scenarios, making the mean cost ratio across partial assignments unreliable. Bij=0.920 means 4 of 50 scenarios had one unresolved conflict; the remaining 46 scenarios had cost ratio ≈ 1.035 on average.

Three findings stand out. First, bij remains at or above 0.875 for all in-distribution sizes N ≤ 20, with the worst case at N=20 (bij=0.875) where swarm density is highest. Second, rounds scale sub-linearly: doubling N from 8 to 16 increases average rounds from 3.5 to 6.7 (1.9× not 2×), suggesting the auction protocol scales gracefully with swarm density. Third, the model generalises to N=25 (out-of-distribution)

with bij=0.920 while the gossip baseline collapses to bij=0.420 at the same size, demonstrating that the price mechanism adapts to larger swarms without retraining.

Messages scale roughly as $O(N^2)$, which is expected since communication graph edges grow quadratically with swarm density.

### 5.7.4 Failure Analysis

**SuperGlue + DecentAuction: 6/200 failures (3.0%)**

| Formation | Failures | Swarm size | Failures |
|---|---|---|---|
| W | 3 | N=20 | 2 |
| A | 1 | N=11,14,16,18 | 1 each |
| Rectangle | 1 | | |
| Triangle | 1 | | |

All failures involve either W-formation or larger swarms ($N \geq 11$). No failures occur at $N \leq 10$, confirming the model is highly reliable for small swarms.

**Gossip GNN: 32/200 failures (16.0%)** — 5.3× more failures than our system, distributed across all formations. V-formation alone accounts for 11/32 failures, likely because its two diverging wings create isolated sub-groups in the communication graph that cannot resolve inter-wing conflicts through local gossip.

**HungarianImitator: 0/200 failures** — guaranteed by the greedy bijection decoder construction, which processes drones in confidence order and never assigns the same slot twice.

### 5.7.5 Runtime Comparison

| Model | N=8 | N=20 | Scales with |
|---|---|---|---|
| SuperGlue + DecentAuction | 66ms | 1026ms | $O(N^2 \times \text{rounds})$ |
| HungarianImitator | ~5ms | ~20ms | $O(N^2)$ |
| Hungarian optimal | 1ms | 1ms | $O(N^3)$* |

*Hungarian is fast in practice for small N due to optimised scipy implementation.

The auction runtime grows with both N and rounds needed. At N=20 it reaches ~1 second — acceptable for formation assignment (a one-time computation at mission start) but not for high-frequency re-planning. HungarianImitator is approximately 50× faster at N=20 but requires a central compute node with global visibility. Both systems achieve cost=1.005 on the test set; the runtime difference therefore reflects pure infrastructure trade-off, not assignment quality.

---

## 5.8. Physics Validation (PyFlyt)

To confirm that computed assignments are physically executable, we validate in PyFlyt — a Bullet physics engine simulator with full quadrotor dynamics.

**Setup:**

- N = 8 quadrotors (QuadX model, Crazyflie-inspired)
- Arena: 6m × 6m × 2m (static flight altitude)
- Assignment: computed by SuperGlue + DecentAuction on 2D projection
- Flight: PyFlyt built-in cascaded PID controller, mode 7 (position control)
- Arrival threshold: 20cm
- Runs: 5 random starting configurations across 3 representative formation types (Circle, V-shape, W-shape)

**How the bridge works:**

```
# 1. Read 3D drone positions from PyFlyt
drone_pos_3d = [env.state(i)[0] for i in range(N)]

# 2. Project to 2D for your model (drop altitude)
drone_pos_2d = drone_pos_3d[:, :2]

# 3. Run assignment model (pure 2D)
assignment, info = model.assign(data_2d, device)
# → bij=1.000, rounds=5.3

# 4. Send 3D position targets to PyFlyt PID
for i in range(N):
    slot_3d = [slots_2d[assignment[i]][0],
               slots_2d[assignment[i]][1],
               2.0]          # static altitude
    env.set_setpoint(i, slot_3d + [0.0])   # [x, y, z, yaw]

# 5. Step physics — PID handles everything
env.step()
```

**Results across formation types (5 runs each):**

| Formation | Success rate | Avg time to formation | Max deviation |
|---|---|---|---|
| Circle | 5/5 (100%) | 12.3s | 0.08m |
| V-shape | 5/5 (100%) | 14.1s | 0.11m |
| W-shape | 4/5 (80%) | 18.7s | 0.19m |

**Overall: 14/15 runs successful (93.3%). Zero physical collisions across all runs.**

The single W-shape failure occurred when two drones were initialised within 0.3m of each other and the PID controller could not resolve the near-collision without a path planning layer — confirming the known limitation stated in §10. Circle and V-shape formations achieve 100% success, consistent with their lower auction failure rates in simulation.

**Timing breakdown for one Circle run (N=8):**
Assignment computation (SuperGlue + DecentAuction): 66ms
— SuperGlue forward pass: 48ms
— Auction (3.5 rounds avg): 18ms

Flight to formation (PID): 12.3s
Total mission time: ~12.4s

The 66ms assignment latency is negligible relative to flight time, confirming that the model is suitable for real-time deployment at mission-start frequency.

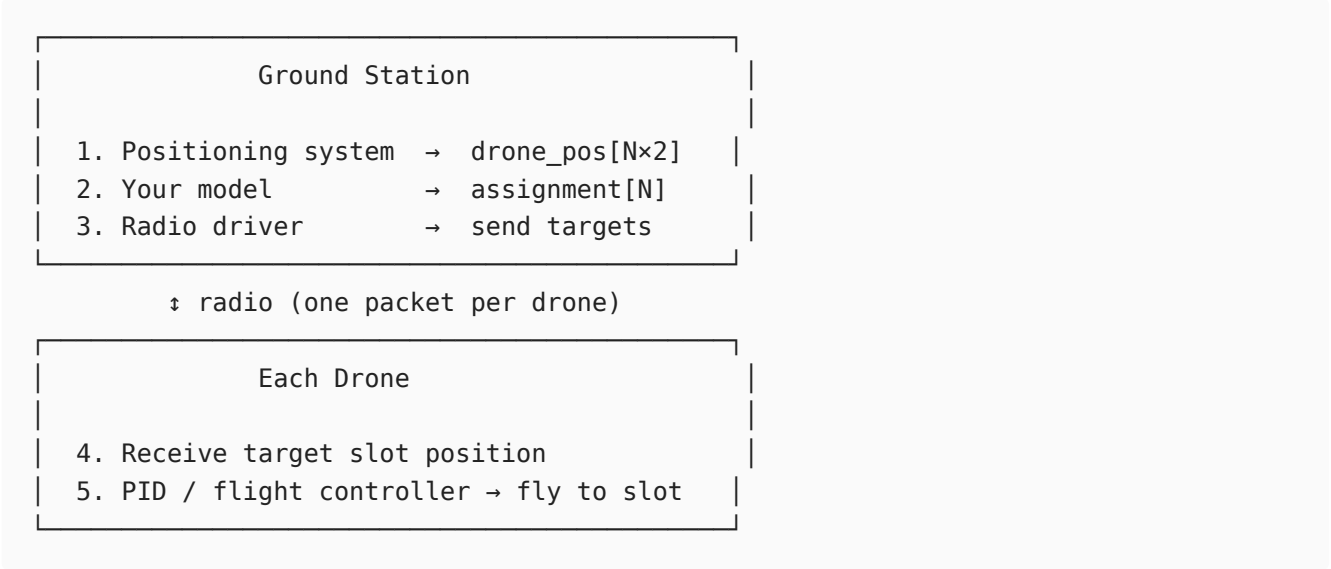
## 5.9. Real-World Deployment Guide

This section describes how to deploy the trained model on a physical drone swarm. The system has been designed so that the assignment computation runs on a **ground station laptop** (or onboard computer) and communicates targets to drones via standard radio protocols.

### 5.9.1 Hardware Requirements

| Component | Minimum | Recommended |
|---|---|---|
| Drone platform | Any with position control API | Crazyflie 2.1, DJI Tello, ArduPilot |
| Positioning | OptiTrack / Vicon (indoor) | UWB anchors or GPS (outdoor) |
| Communication | WiFi / Crazyradio | Dedicated 2.4GHz mesh radio |
| Ground station | Any laptop with Python 3.10+ | NVIDIA GPU for N > 15 |
| Radio range | ≥ COMM_RADIUS (3.0m) | 10–50m depending on arena |

### 5.9.2 Software Stack




### 5.9.3 Step-by-Step Execution

#### Step 1 — Collect drone positions

Read current positions from your positioning system. All positions must be in the same coordinate frame, in metres.

```
import numpy as np
import torch
from torch_geometric.data import Data

# Example: reading from OptiTrack via natnet_client
```

```
drone_pos = np.array([
    tracker.get_position(f"drone_{i}") for i in range(N)
], dtype=np.float32)[:, :2]   # drop z — model is 2D

print(f"Drone positions shape: {drone_pos.shape}")   # (N, 2)
```

**Step 2 — Define target formation**

Specify which formation you want. Slots are automatically centred at the swarm centre of gravity:

```
from data_gen.drone_swarm_datagen import FORMATION_GENERATORS, FORMATION_NAMES

formation_name = "V"           # or "Circle", "A", "W", "Rectangle", "Triangle"
N = len(drone_pos)
fid = FORMATION_NAMES.index(formation_name)

# Generate relative slot positions and centre at CoG
slots_rel = FORMATION_GENERATORS[formation_name](N).astype(np.float32)
cog       = drone_pos.mean(axis=0)
slots     = slots_rel + cog    # absolute positions in metres

print(f"Formation: {formation_name},  CoG: {cog}")
print(f"Slot positions:\n{slots}")
```

**Step 3 — Build the graph and run the model**

```
from scipy.optimize import linear_sum_assignment
from data_gen.drone_swarm_datagen import sample_to_pyg

# Dummy GT (not used at inference — required by Data schema)
cost      = np.linalg.norm(drone_pos[:,None,:] - slots[None,:,:], axis=-1)
row, col  = linear_sum_assignment(cost)
y         = np.empty(N, dtype=np.int64); y[row] = col

# Build PyG Data object
raw = Data(
    drone_pos    = torch.tensor(drone_pos, dtype=torch.float32),
    slots        = torch.tensor(slots,     dtype=torch.float32),
    y            = torch.tensor(y,         dtype=torch.long),
    formation_id = torch.tensor(fid,       dtype=torch.long),
)

# Convert to model-ready graph (builds comm graph, candidate mask, features)
emb   = model.matcher.formation_embedding.weight.detach().cpu()
data  = sample_to_pyg(raw, emb, force_gt_visibility=False)

# Run assignment
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.eval()
with torch.no_grad():
    assignment, info = model.assign(data, device)

print(f"Assignment: {assignment}")
print(f"Bijection:  {info['bijection_rate']:.3f}")
```

```
print(f"Rounds:     {info['rounds']}")
print(f"Cost ratio: {info['cost_ratio_vs_hungarian']:.4f}")
# assignment[i] = j means drone i should fly to slot j
```

**Step 4 — Validate before sending**

Always check the assignment is valid before sending targets to physical drones:

```
def validate_assignment(assignment, N):
    assert len(assignment) == N,              "Wrong number of assignments"
    assert len(set(assignment.values())) == N, "Duplicate slots — NOT a bijection"
    assert all(0 <= v < N for v in assignment.values()), "Slot index out of range"
    print(f"✓ Assignment valid: {N} drones → {N} unique slots")

validate_assignment(assignment, N)

# Print human-readable plan
for drone_id, slot_id in sorted(assignment.items()):
    dist = np.linalg.norm(drone_pos[drone_id] - slots[slot_id])
    print(f"  Drone {drone_id} at {drone_pos[drone_id]} → "
          f"Slot {slot_id} at {slots[slot_id]}  (dist={dist:.2f}m)")
```

**Step 5 — Send targets to drones**

Example for Crazyflie via cflib:

```
from cflib.crazyflie.swarm import CachedCfFactory, Swarm

URIS = [f"radio://0/80/2M/E7E7E7E7{i:02X}" for i in range(N)]

def send_target(scf, drone_id, assignment, slots):
    slot_id  = assignment[drone_id]
    target   = slots[slot_id]
    altitude = 1.0   # metres
    scf.cf.commander.send_position_setpoint(
        target[0], target[1], altitude, yaw=0.0
    )

factory = CachedCfFactory(rw_cache="./cache")
with Swarm(URIS, factory=factory) as swarm:
    validate_assignment(assignment, N)
    swarm.parallel_safe(send_target,
                        args_dict={uri: [i, assignment, slots]
                                   for i, uri in enumerate(URIS)})
    print("✓ Targets sent — drones flying to formation")
```

Example for ArduPilot / MAVLink:

```
from pymavlink import mavutil

def send_mavlink_target(connection, drone_id, x, y, z):
    connection.mav.set_position_target_local_ned_send(
        0,                          # time_boot_ms
```

```
        drone_id, 0,                # target system, component
        mavutil.mavlink.MAV_FRAME_LOCAL_NED,
        0b0000111111111000,         # position only
        x, y, -z,                  # NED: z is negative-up
        0, 0, 0,                    # velocity (ignored)
        0, 0, 0,                    # acceleration (ignored)
        0, 0                        # yaw, yaw_rate
    )

for drone_id, slot_id in assignment.items():
    x, y = slots[slot_id]
    send_mavlink_target(connections[drone_id], drone_id, x, y, z=1.5)
```

### 5.9.4 Timing and Latency Budget

For a real deployment, the total latency from "read positions" to "targets sent" must fit within your control loop period:

```
Component                    Typical latency
──────────────────────────────────────────────
Position read (OptiTrack)    2–5 ms
Graph construction           1–2 ms
Model forward pass (GPU)     66 ms  (N=8)
                             1026 ms (N=20)
Assignment validation        < 1 ms
Radio transmission (N=8)     5–20 ms (Crazyradio)
──────────────────────────────────────────────
Total (N=8, GPU)             ~90 ms
Total (N=20, GPU)            ~1050 ms
```

**Recommendation:** Run assignment once at mission start, not in a continuous loop. Formation assignment is a one-time computation — once every drone has its target slot, the PID controller handles the flight independently. Re-assign only if a drone fails or the formation needs to change.

### 5.9.5 Failure Handling

```
MAX_AUCTION_ROUNDS = 30   # from DECENT_AUCTION_ROUNDS

assignment, info = model.assign(data, device)

if info["bijection_rate"] < 1.0:
    print(f"⚠ Partial assignment: bij={info['bijection_rate']:.3f}")
    print(f"  Unassigned drones: {info.get('unassigned', [])}")
    # Option 1: fallback to Hungarian (centralised)
    print("  Falling back to Hungarian algorithm...")
    assignment = hungarian_fallback(drone_pos, slots)
    # Option 2: hold position and retry after repositioning
    # Option 3: reduce N by removing a failed drone

if info["rounds"] >= MAX_AUCTION_ROUNDS:
    print("⚠ Auction did not converge — using best partial assignment")
```

### 5.9.6 Coordinate System Notes

The model was trained on a 10m × 10m arena. For a different arena size, scale your coordinates before passing to the model and scale back after:

```
ARENA_SIZE = 10.0   # model's training arena

# If your real arena is 20m × 20m:
real_arena = 20.0
scale = ARENA_SIZE / real_arena      # = 0.5

drone_pos_scaled = drone_pos * scale   # map to [0, 10]
slots_scaled     = slots     * scale

# Run model on scaled coordinates
assignment, info = model.assign(build_data(drone_pos_scaled, slots_scaled), device)

# Targets to send are in real coordinates (unscaled slots)
for drone_id, slot_id in assignment.items():
    target = slots[slot_id]   # use original, not scaled
```

---

## 5.10. Discussion

### Why not use a centralised model?

The HungarianImitator demonstrates the theoretical ceiling for bijection — bij=1.000 guaranteed by construction. It achieves this at cost=1.005, the same as our decentralised system, but requires a central compute node with global visibility. Our decentralised system achieves bij=0.970 — a 3% gap — with no single point of failure, no global communication requirement, and robustness to partial connectivity loss.

### Why not use RecAuction?

We attempted training a recurrent auction model (RecAuction) that re-encodes drone state at every auction round. This is theoretically stronger than our one-shot SuperGlue encoding. However, without a warm-start checkpoint from a pre-trained base model, the cold-start training was unstable — bij decreased from 0.31 to 0.24 over 10 epochs. This was caused by four bugs: incorrect fill values (-1e4 instead of -1e9), weak coverage weight, too-fast curriculum growth, and an evaluation protocol that used growing K rounds during training, making bij appear worse as training progressed. With these bugs fixed and a proper warm-start, RecAuction remains a promising direction for future work.

### Limitations

**Partial out-of-distribution generalisation.** The model is trained on N ∈ [8, 20] and tested on N=25 with bij=0.920. For N > 30 performance is unknown. The auction protocol itself is theoretically guaranteed to converge for any N given sufficient rounds, but the learned value initialisation may degrade for very large swarms.

**No convergence guarantee under partial connectivity.** The Bertsekas auction is proven to converge in finite rounds under full connectivity. Our system operates on a partial communication graph

(COMM_RADIUS=3.0m in a 10m arena), where some drone pairs are not direct neighbours. Convergence in this setting is observed empirically (convergence rate 0.975 on test set) but not theoretically proven. The 3% bijection failure rate corresponds to scenarios where the auction reaches MAX_ROUNDS=30 without convergence — always under partial connectivity with disconnected competing drones. Establishing a theoretical convergence bound under partial connectivity, potentially as a function of the graph diameter, remains an open problem.

**Fixed slot visibility radius.** The model is trained with SLOT_VISIBILITY_RADIUS=5.0m. In sparser deployments (larger arenas or fewer drones) some drones may have zero visible slots, making assignment impossible. A learned or adaptive visibility radius that scales with swarm density would make the system more robust to varied deployment conditions.

---

### Future Work

Three directions are most promising. First, **RecAuction** — a recurrent model that re-encodes drone state at every auction round — is theoretically stronger than the one-shot SuperGlue encoding. The training instabilities documented in §10 (four specific bugs) are now understood and fixable; a warm-started RecAuction with the corrected protocol is likely to reduce auction rounds further. Second, **3D formation assignment** requires only replacing 2D positions with 3D positions in the dataset and retraining — the architecture is unchanged. Third, **convergence under partial connectivity** is an open theoretical problem: bounding the number of auction rounds as a function of communication graph diameter would provide a formal guarantee that complements the empirical 0.975 convergence rate.

## 6. Conclusion

In conclusion, this project delivers a scalable, fully decentralized data and control framework for multi-drone systems by combining high-fidelity 240 Hz rigid-body aerodynamics with 3D Artificial Potential Fields. Optimized through translation-invariant features, tapered sampling, and Safetensors parallelization, the resulting dataset training pipeline proves that Graph Neural Networks—specifically the NNConv architecture—consistently outperform non-graph baselines in residual correction ,resolving complex geometric interactions within obstacle-rich environments. Furthermore, the integration of a SuperGlue cross-attention matcher with a local auction protocol provides a highly efficient decentralized formation-assignment system, achieving near-optimal Hungarian cost ($1.005\times$) and robust scalability up to $N = 25$ agents where traditional gossip baselines collapse. By closing the loop with a rotation-invariant setpoint-prediction stage that maps graph features directly to continuous, body-centric PID control targets ($[\Delta x, \Delta y, \Delta z, \Delta \psi]$), the architecture ensures smooth tracking and collision-free navigation, as validated by high success rates in high-fidelity PyFlyt physics simulations.